Respuestas del 2do parcial

Defensa oral del frontend · Banco de 50 preguntas

StockFlow — Inventario, compras internas y movimientos de almacén (PA-06)

Programación Aplicada 2026-2 · React 19 · TypeScript 6 · Vite 8 · React Router 8

Código analizado	Rama `Hector`, commit `a8a11e2`
Fecha	17 de octubre de 2026
Contenido	50 preguntas con explicación del concepto, respuesta, ubicación en el código, demostración y repregunta · 5 desafíos prácticos · checklist · pendientes conocidos · glosario
Pruebas del frontend	30 archivos de prueba, 107 pruebas, todas en verde (`npm run test:run`)

Las rutas de archivos cuelgan de `webstorm/stockflow-frontend/` salvo que se indique otra. Los números de línea corresponden al commit indicado; si el código cambia, busca el archivo o la función por su nombre (Ctrl+Shift+N y Ctrl+F12 en WebStorm).

Índice

Cómo usar esta guía 4

Mapa del proyecto, flujo y preparación de la demo 7

Bloque A — Arquitectura frontend, Vite, TypeScript y Router 9

01 (C) — Arquitectura real del frontend y recorrido entre carpetas 9
02 — Qué resuelve Vite: npm install, npm run dev y npm run build 10
03 — Arranque: index.html → main.tsx → App → Router → pantalla 11
04 — interface, type y tipos en tiempo de ejecución 12
05 (C) — Por qué no copiar las @Entity JPA: contratos TypeScript 13
06 (C) — Recorrido de una URL dentro de React Router 14
07 — Link, NavLink, navegación declarativa y menú activo 15
08 — Layout, Page, Feature y componente UI reutilizable 16
09 — import.meta.env, variables VITE_* y secretos 17
10 — Flujo unidireccional: props hacia abajo, callbacks hacia arriba 18

Bloque B — Componentes, estado, formularios y UI 19

11 (C) — Input controlado con useState, value y onChange 19
12 — Por qué el formulario guarda números como string 20
13 — Eventos tipados y preventDefault() 21
14 — Validación de UI, del backend y constraints de la base 21
15 (C) — El hijo comunica acciones al padre con props callback 22
16 — map(), filter(), find() y key 23
17 — Estado fuente y estado derivado 24
18 — Pantalla con loading, error, empty y success 25
19 — Orden: datos → búsqueda/filtros → paginación → tabla 26
20 — Modal/ConfirmDialog en lugar de window.confirm 27

Bloque C — HTTP, services y CRUD fullstack 28

21 (C) — Flujo completo de un GET de React a Spring Boot y de vuelta 28
22 — Por qué fetch() no rechaza ante un 404 o un 500 29
23 (C) — CORS: por qué Postman funciona y React falla 30
24 — Separación entre apiClient, services, hooks y componentes 31
25 (C) — POST línea por línea: de Guardar a la fila nueva 32
26 — GET/{id}: seleccionar para editar y volver a consultar 33
27 (C) — PUT completo y un formulario para crear y editar 34
28 (C) — DELETE: confirmación, 204 y sincronización de la tabla 35
29 — Cómo interpreta el frontend 400, 404, 409 y 500 36
30 — Actualizar estado local o hacer refetch 37
31 — Promise, async/await, try/catch/finally 37
32 — Diagnóstico ordenado: Swagger funciona y React no 38

Bloque D — Relación 1:N, asincronía, hooks y UI avanzada 40

33 (C) — Categoría 1:N Producto en el frontend y el papel de categoriaId 40
34 — Select controlado de categoría y conversión string → number 41
35 (C) — Cambiar la categoría de un producto con PUT 42
36 — Mostrar el nombre de la categoría sin duplicarlo 43
37 — Filtrar productos por categoría: frontend o backend 44
38 (C) — useEffect como sincronización con sistemas externos 44
39 (C) — Cleanup, AbortController y respuestas obsoletas 45

40 — Effects que corren dos veces con StrictMode 47
41 — Qué resuelven useAsyncList<T>, useCategorias y useProductos 47
42 — Componentes UI compartidos, useMemo, tokens y accesibilidad 48

Bloque E — Testing, build, producción y defensa integral 50
43 (C) — Estrategia de testing por niveles 50
44 — Vitest, React Testing Library y userEvent 51
45 — Cuándo mockear la API y cuándo integrar de verdad 52
46 — ErrorBoundary vs manejo de errores de fetch 53
47 (C) — npm run build, dist, npm run preview y .env por modo 54
48 — BrowserRouter en producción: fallback SPA y CORS productivo 55
49 — El Dashboard y por qué sus métricas son estado derivado 56
50 (C) — Flujo fullstack completo sin saltos 57

Los 5 desafíos prácticos 59

Señales de alerta y checklist final 61

Pendientes conocidos del frontend (lo que el docente podría señalar) 62

Glosario de términos técnicos 63

Cómo usar esta guía

- Esta guía responde las 50 preguntas y los 5 desafíos del **Banco de 50 preguntas — Defensa oral del frontend** (Programación Aplicada 2026-2, segundo parcial) usando el código real del frontend de StockFlow.
- Cada pregunta tiene cinco partes:
  - **Para entender el concepto** (recuadro verde): la idea general explicada de forma simple, sin depender del código. Léela primero si el tema te resulta nuevo.
  - **Qué responder**: la respuesta que darías en la defensa, cubriendo todo lo que el docente marca como "Debe demostrar".
  - **Muéstralo en el código** (recuadro gris): el archivo y la línea que debes abrir en WebStorm.
  - **Demuéstralo** (recuadro punteado, cuando aplica): algo que puedes ejecutar o mirar en DevTools.
  - **Repregunta de presión** (recuadro amarillo): la repregunta del docente con su respuesta.
- Los términos técnicos (CORS, HMR, payload, race condition…) están explicados en el **Glosario** al final.
- **No la memorices.** El banco dice: "*una definición memorizada sin poder ubicarla, recorrerla y justificarla en el propio código se considera respuesta incompleta*". Úsala así: lee la pregunta, intenta responder en voz alta sin mirar, abre el archivo indicado en WebStorm y recién entonces compara con la guía.
- Las preguntas marcadas **CRÍTICA** son 17. Fallar dos o más obliga a una repregunta adicional: son las que más conviene dominar.

Las 5 exigencias de cada respuesta

Exigencia	Qué debes hacer
Definir	Explicar el concepto con palabras propias, sin leer.
Ubicar	Mostrar en qué archivo, carpeta, componente, hook, service o configuración aparece.
Recorrer	Explicar qué sucede antes y después de ese punto dentro del flujo.
Justificar	Explicar por qué se usó así y qué alternativa existía.
Predecir	Decir qué ocurriría si se elimina, cambia, duplica o falla esa pieza.

Cómo se toma el examen

- Se eligen **10 a 12 preguntas al azar**, incluyendo siempre al menos una de: arquitectura/Vite, React/TypeScript, formularios/estado, HTTP/CRUD, relación padre-hijo (Cliente-Vehículo en el examen; Categoría-Producto en StockFlow), asincronía/hooks y testing/build.
- En 3 o 4 preguntas te pedirán **abrir WebStorm** y señalar el archivo y el código exacto.
- En al menos una tendrás que **dibujar el flujo completo**: usuario → React → hook/service → HTTP → Spring Boot → respuesta → estado → re-render.
- No se acepta "React lo hace" ni "Vite lo hace": hay que nombrar la API, el componente o la responsabilidad (por ejemplo `createRoot`, `useState`, `fetch`, `response.ok`, `Outlet`) y el efecto que produce.
- El docente cambiará Cliente/Vehículo por las entidades reales del proyecto: en StockFlow, **Categoría/Producto**.

Calificación

Puntos	Nivel	Criterio
0	No domina	No puede explicarlo, confunde conceptos o no puede localizarlo en su código.
1	Parcial	Conoce la idea, pero falla al recorrer el flujo, justificar la decisión o relacionarla con su proyecto.
2	Dominio	Explica, localiza, conecta, justifica y predice consecuencias técnicas con seguridad.

Escala del banco completo (100 puntos): **85-100** preparado · **70-84** defendible con vacíos · **50-69** preparación insuficiente · **menos de 50** no domina su frontend.

Fórmula para responder cualquier pregunta

1. Defino con mis palabras (1-2 frases). 2. Digo dónde está en StockFlow y lo abro. 3. Recorro qué pasa antes y después. 4. Justifico por qué así y qué alternativa descarté. 5. Predigo qué fallaría si lo cambio o lo quito.

Traducción del examen (ParkFlow360) a StockFlow

El banco usa el proyecto del docente. En la defensa responde siempre con los nombres de StockFlow:

En el examen (ParkFlow360)	En StockFlow
Cliente (entidad padre)	**Categoría** — `features/categorias/`
Vehículo (entidad hija)	**Producto** — `features/productos/`
`clienteId`	`categoriaId` (FK `producto.categoria_id` en PostgreSQL)
Placa única	Código de producto único (RN-06) y código de categoría único
"Cambiar el propietario de un vehículo"	Cambiar la categoría de un producto (con la regla 422: no se puede mover a una categoría inactiva)
`ClienteForm`, `VehiculoForm`	`CategoriaForm`, `ProductoForm`
`ClienteTable`, `VehiculoTable`	`CategoriaTable`, `ProductoTable`
`clienteService`, `vehiculoService`	`categoriaService`, `productoService`
`useClientes`, `useVehiculos`	`useCategorias`, `useProductos`
`VITE_API_BASE_URL`	`VITE_API_URL`
`/clientes`, `/vehiculos` (rutas del navegador)	`/categorias`, `/productos`

Si el docente cambia la entidad

El banco dice que el docente puede cambiar los nombres "para impedir respuestas memorizadas". Si pregunta con otra pantalla de StockFlow, estos son los equivalentes:

Tema (preguntas)	Con Ubicación (pantalla Almacenes)	Con Kardex (pantalla Movimientos)

Formulario y select (11, 12, 34)	`UbicacionForm.tsx:85`: `<Select name="tipo">` con `TIPOS_UBICACION`. El tipo es un enum de texto, así que no se convierte a número; se valida con `ubicacionValidation`.	El producto se elige en un select (`MovimientosPage.tsx:126-128`) y viaja en la URL (`?producto=ID`); se convierte con `Number(params.get('producto'))` (`:39`).
POST (25)	`UbicacionForm.tsx:39-51`: `handleSubmit` → `preventDefault` → validar → `ubicacionService.crear` → `onSaved`. 409 si el código existe o si ya hay un almacén central (`AlmacenesPage.test.tsx:54`).	No aplica: la pantalla es de sólo lectura.
GET, loading, error (18, 21)	`AlmacenesPage.tsx:26-27`: `useAsyncList(ubicacionService.listar)` y las existencias.	`useKardex` (`MovimientosPage.tsx:42`): paginado en el servidor.
PUT y DELETE (26-28)	**No existen** en la API (RF-07); la pantalla lo dice (`AlmacenesPage.tsx:49`). Responde con Categoría/Producto y explica que es un pendiente del backend.	No aplica: el kardex es un histórico que no se edita ni se borra (RN-01, trigger de la migración V3).
Paginación (19, 37)	En el cliente, con `paginar()`, como Productos.	En el servidor: `PaginaResponse` trae `contenido`, `pagina`, `tamano`, `totalElementos` y `totalPaginas`, y `Pagination` usa esos totales (`MovimientosPage.tsx:191-194`). Se envía `pagina - 1` porque la API cuenta desde 0 (`:42`).
Race conditions (39)	—	`useKardex` y su prueba (`useKardex.test.ts:27`).

Mapa del proyecto

Qué	Dónde (dentro de `src/`)
Entrada	`../index.html` (div `#root`) → `main.tsx` (createRoot, StrictMode, AppErrorBoundary, BrowserRouter)
Composición global	`app/App.tsx` (providers: tema, sesión demo, toasts) · `app/navigation.ts` (menú) · `app/AppErrorBoundary.tsx`
Rutas	`routes/AppRouter.tsx` (rutas lazy, 404)
Estructura persistente	`layouts/MainLayout.tsx` (Sidebar + Header + `<Outlet>`)
Configuración	`config/env.ts` (lee `VITE_API_URL`) · `vite-env.d.ts` (tipo de la variable) · `../.env.development`, `../.env.production` (`/api`), `../.env.example` · `../nginx.conf` (producción: fallback SPA y proxy de `/api/`)
Capa HTTP	`api/apiClient.ts` (`apiFetch`, `ApiError`, `mensajeDeError`) · `api/describirError.ts` (400/404/409/422/500 → mensaje)
Módulo padre: categorías	`features/categorias/` → `models/`, `types/`, `services/categoriaService.ts`, `hooks/useCategorias.ts`, `components/CategoriaForm.tsx`, `CategoriaTable.tsx`, `pages/CategoriasPage.tsx`, `utils/categoriaValidation.ts`
Módulo hijo: productos	`features/productos/` → misma estructura: `Producto.ts`, `ProductoFormData.ts`, `ProductoRequests.ts`, `productoService.ts`, `useProductos.ts`, `ProductoForm.tsx`, `ProductoTable.tsx`, `ProductosPage.tsx`
Inventario (lectura)	`features/inventario/` → existencias, kardex paginado (`hooks/useKardex.ts`), resumen para el dashboard
Otras pantallas	`features/ubicaciones/` (Almacenes, real) · `features/reportes/` (CSV) · `features/auth/`, `usuarios/`, `configuracion/` (demo, sin backend)
UI compartida	`components/ui/` (Button, Field, SearchInput, SelectFilter, Pagination, Modal, ConfirmDialog, DataTable, StatusBadge, EmptyState, Alert, toast…) · `components/common/` (Sidebar, Header, Breadcrumbs…)
Utilidades y hooks genéricos	`shared/hooks/useAsyncList.ts` · `shared/utils/paginar.ts`, `isAbortError.ts`, `formato.ts`
Estilos	`styles/tokens.css` (design tokens), `base.css`, `layout.css`, `components.css`, `pages.css`
Pruebas	`*.test.ts(x)` junto a cada archivo · `test/setup.ts`, `test/renderWithProviders.tsx`, `test/fixtures.ts` · 30 archivos, 107 pruebas

Flujo que debes poder dibujar

Usuario hace clic en *Guardar* → `ProductoForm` (`handleSubmit`: `preventDefault` → `validarProducto` → `aPayload`) → `productoService.crear` → `apiFetch` (URL base + headers JSON + `fetch`) → **HTTP POST /api/productos** → [backend: CORS → DispatcherServlet → ProductoController → @Valid → ProductoService → PostgreSQL] → **201 + JSON** → `response.ok` → `response.json()` → `onSaved(producto)` → `ProductosPage.handleSaved` → `productos.setData([...prev, saved])` → React vuelve a renderizar → aparece la fila nueva.

Preparar la demostración (antes de entrar)

1. **Base de datos:** en la raíz del repositorio, `docker compose up -d` (PostgreSQL 16 en el puerto 5432).
2. **Backend:** en IntelliJ ejecuta `StockFlowApplication` (o `mvn spring-boot:run` en `intellij/stockflow-backend`). Comprueba `http://localhost:8080/api/health`.
3. **Frontend:** en `webstorm/stockflow-frontend`, `npm install` (sólo la primera vez) y `npm run dev` → `http://localhost:5173`.
4. **DevTools:** abre F12 → pestaña **Network** (filtro *Fetch/XHR*) y **Console**. Ahí demuestras cada petición real.
5. **Pruebas:** `npm run test:run` (107 pruebas en verde) o `npm test` en modo observación.
6. **Build:** `npm run build` genera `dist/`; `npm run preview` lo sirve en local (puerto 4173) y reenvía `/api` al backend, así se puede navegar el build con datos reales.

Códigos HTTP vistos desde el frontend

Código	Qué significa	Cómo reacciona StockFlow
200 OK	GET o PUT exitoso, con cuerpo JSON	`apiFetch` devuelve `response.json()` y la Page actualiza su estado
201 Created	POST que creó el recurso	El objeto devuelto (con su `id`) se agrega a la lista
204 No Content	DELETE exitoso, sin cuerpo	`apiFetch` no intenta leer JSON (`apiClient.ts:64-66`) y la Page quita la fila
400 Bad Request	Datos inválidos (Bean Validation)	Título "Revisa los datos ingresados"; los `fieldErrors` se pintan junto a cada input
404 Not Found	El recurso no existe (otro usuario lo borró)	"El registro ya no existe" y se recarga la lista (`reload()`)
409 Conflict	Choca con datos existentes: código repetido, categoría con productos, producto con stock	Mensaje del backend; la fila **no** se quita
422 Unprocessable	Regla de negocio violada (mover un producto a una categoría inactiva)	"La operación no cumple una regla del inventario" + el detalle del backend
500	Fallo interno del servidor	"El servidor tuvo un problema"; no se finge éxito
(sin respuesta)	Error de red: backend apagado, CORS, URL mal escrita	`fetch` lanza `TypeError` → "No se pudo conectar con el backend…" (`apiClient.ts:73-75`)

Bloque A — Arquitectura frontend, Vite, TypeScript y Router

Objetivo del bloque: comprobar que entiendes cómo nace, se organiza y navega una aplicación React profesional.

Pregunta 01 · CRÍTICA

> *Dibuja la arquitectura real de tu frontend y explica el recorrido entre sus carpetas.*

Para entender el concepto

La **arquitectura** de un frontend es la forma en que repartes responsabilidades entre carpetas y archivos para que cada pieza haga una sola cosa. Piensa en un restaurante: el mozo (componente) toma el pedido, no cocina; la cocina (service) prepara, no atiende mesas; el proveedor (backend) entrega los ingredientes. Si el mozo también cocinara y fuera al mercado, cualquier cambio rompería todo.

La regla clave es la **dirección de las dependencias**: las pantallas usan hooks, los hooks usan services, los services usan el cliente HTTP, y el cliente HTTP usa la configuración. Nunca al revés: el cliente HTTP no sabe que existe una pantalla de productos.

Qué responder

- El frontend de StockFlow es una **SPA** en React + TypeScript construida con Vite y organizada **por features** (capacidades de negocio), no por tipo de archivo.
- Carpetas y responsabilidad:
  - `app/`: composición global (providers de tema, sesión demo y toasts), menú (`navigation.ts`) y ErrorBoundary.
  - `routes/`: qué página corresponde a cada URL (`AppRouter.tsx`).
  - `layouts/`: la estructura que no cambia entre pantallas (Sidebar + Header + `Outlet`).
  - `pages/` y `features/*/pages/`: cada pantalla compone componentes y decide qué hacer.
  - `features/<modulo>/`: todo lo de un módulo de negocio junto (`models`, `types`, `services`, `hooks`, `components`, `pages`, `utils`).
  - `components/ui/`: piezas visuales genéricas sin lógica de negocio (Button, Modal, Pagination…). `components/common/`: piezas del "marco" (Sidebar, Header).
  - `api/`: la única puerta HTTP (`apiFetch`) y la traducción de errores.
  - `config/`: lectura de variables de entorno. `shared/`: hooks y utilidades genéricas. `styles/`: design tokens y CSS.
- **Organización física vs responsabilidad:** la carpeta es dónde vive el archivo; la responsabilidad es qué decide. `ProductoForm` vive en `features/productos/components` porque pertenece al negocio de productos; `Button` vive en `components/ui` porque no sabe nada de productos.
- **Dirección del flujo:** `ProductosPage` → `useProductos` → `productoService` → `apiFetch` → `config/env`. Los componentes UI no importan nada de `features/`; la tabla no importa services. Así se evita que "cualquier componente llame a cualquier cosa".

Muéstralo en el código

- Árbol de `src/` en WebStorm: `app`, `routes`, `layouts`, `pages`, `features`, `components/ui`, `components/common`, `api`, `config`, `shared`, `styles`, `test`.
- Cadena real: `src/features/productos/pages/ProductosPage.tsx:42` (`useProductos()`) → `features/productos/hooks/useProductos.ts:5` → `features/productos/services/productoService.ts:6-9` → `src/api/apiClient.ts:52-68` → `src/config/env.ts:2`.
- La tabla no importa services: `features/productos/components/ProductoTable.tsx:1-12` (sólo UI, modelos y utilidades).

Si muevo toda la lógica HTTP dentro de un componente visual, ¿el proyecto puede funcionar? Aunque funcione, ¿qué problema de acoplamiento aparece?

Sí, puede funcionar: `fetch` se puede llamar desde cualquier componente. Pero el componente quedaría **acoplado** a la URL, a los headers, al formato de errores y al backend: no se podría reutilizar con otros datos, habría que repetir la verificación de `response.ok` en cada pantalla, probarlo exigiría simular `fetch` en cada test y un cambio de URL o de librería (por ejemplo pasar a Axios) obligaría a editar todas las pantallas. Con `apiClient` + services ese cambio se hace en un solo lugar.

Pregunta 02

> *Explica qué problema resuelve Vite y qué ocurre en npm install, npm run dev y npm run build.*

Para entender el concepto

El navegador sólo entiende HTML, CSS y JavaScript. Nosotros escribimos **TypeScript y JSX** (TSX), importamos librerías de `node_modules` y CSS desde archivos de código. Alguien tiene que traducir todo eso: esa es la tarea de una **build tool**.

Hay tres herramientas distintas que conviene no confundir: **npm** es el gestor de paquetes (descarga librerías), **Vite en modo dev** es un servidor de desarrollo que traduce los archivos al vuelo y recarga sólo lo que cambió (HMR), y **Vite build** es el empaquetador que genera los archivos finales optimizados para producción.

Qué responder

- **Vite** resuelve dos problemas: en desarrollo, levantar la app al instante y reflejar cada cambio sin recargar toda la página; en producción, generar un paquete pequeño y optimizado.
- `npm install`: el **gestor de paquetes** lee `package.json` (qué librerías y rangos de versión) y `package-lock.json` (las versiones exactas) y descarga todo a `node_modules/`. No compila nada.
- `npm run dev` → `vite`: levanta el **servidor de desarrollo** en `http://localhost:5173`. Transforma cada `.ts/.tsx` a JavaScript cuando el navegador lo pide y aplica **HMR** (Hot Module Replacement): al guardar un archivo sólo se reemplaza ese módulo y el estado de la pantalla suele conservarse.
- `npm run build` → `tsc -b && vite build`: primero `tsc` **verifica los tipos** (si hay un error de TypeScript el build se detiene); luego Vite **empaqueta**: une módulos, elimina código no usado, minifica, separa en trozos (code splitting) y deja todo en `dist/`.
- Vite **no reemplaza** a React ni a TypeScript: React dibuja la interfaz, TypeScript define y verifica tipos, Vite sólo transforma y sirve/empaqueta los archivos.

- En StockFlow el servidor de desarrollo está fijado en el puerto 5173 con `strictPort: true`: si el puerto está ocupado, Vite se detiene con un error claro en vez de usar 5174 (que el CORS del backend no permite).

Muéstralo en el código

- `package.json` → `"scripts"`: `dev`, `build`, `lint`, `preview`, `test`, `test:run`.
- `vite.config.ts:5-12`: plugin de React y `server.port: 5173` + `strictPort`.
- `package-lock.json` (versiones exactas) y `node_modules/` (ignorado por Git).

Demuéstralo

Con `npm run dev` abierto, cambia un texto de `pages/InicioPage.tsx` y guarda: la pantalla se actualiza sin recargar (HMR). Luego ejecuta `npm run build` y muestra la carpeta `dist/assets` con varios archivos `.js` (uno por página lazy).

Si eliminas node_modules pero conservas package.json y package-lock.json, ¿cómo recuperas el proyecto y por qué?

Con `npm install` (o `npm ci`, que instala exactamente lo del lock y falla si no coincide). `node_modules` es un resultado reproducible: `package.json` dice qué dependencias hacen falta y `package-lock.json` fija las versiones exactas, así que se reconstruye idéntico. Por eso `node_modules` no se sube a Git.

Pregunta 03

Explica el arranque real del frontend: index.html → main.tsx → App.tsx → Router → pantalla.

Para entender el concepto

Una SPA empieza con un HTML casi vacío: sólo un `<div id="root">` y un `<script>`. Todo lo que ves después lo construye React con JavaScript. El HTML es el "terreno vacío"; React es el que levanta la casa dentro de ese terreno.

Qué responder

1. `index.html` se descarga primero. Tiene `<div id="root"></div>` y `<script type="module" src="/src/main.tsx">`. (StockFlow además aplica el tema claro/oscuro antes del primer pintado para evitar un destello.)
2. `main.tsx` busca ese div con `document.getElementById('root')`, crea la raíz de React con `createRoot(...)` y llama a `.render(...)` con el árbol inicial: `StrictMode` → `AppErrorBoundary` → `BrowserRouter` → `App`. Aquí también se importan los CSS globales y la fuente.
3. `App.tsx` sólo envuelve con los providers globales (tema, sesión demo, toasts) y delega en `AppRouter`.
4. `AppRouter` compara la URL con sus `<Route>` y elige qué renderizar: para `/productos`, el layout `MainLayout` y dentro de su `Outlet`, `ProductosPage`.
5. `ProductosPage` se monta, su hook dispara la petición al backend y la pantalla se completa cuando llegan los datos.

HTML base vs UI dinámica: lo que está en `index.html` existe antes de React; todo lo demás (menú, tablas, formularios) lo crea React dentro de `#root` y lo actualiza sin recargar la página.

Muéstralo en el código

- `index.html:21-22`: `<div id="root">` y el script de `/src/main.tsx`.
- `src/main.tsx:15-23`: `createRoot(document.getElementById('root')!).render(...)`.
- `src/app/App.tsx:6-16`: providers y `<AppRouter />`.
- `src/routes/AppRouter.tsx:39-52`: rutas hijas del layout.

Si main.tsx apunta a un id que no existe en index.html, ¿qué esperas que ocurra?

`getElementById` devuelve `null`. El `!` de TypeScript sólo silencia el aviso en compilación, no evita el problema: en ejecución `createRoot(null)` lanza un error ("Target container is not a DOM element"), la app no se monta y la página queda en blanco con el error en la consola. Ni siquiera el ErrorBoundary puede mostrarse, porque React nunca llegó a renderizar.

Pregunta 04

Diferencia interface, type y los tipos reales en tiempo de ejecución usando Cliente y Vehículo (en StockFlow: Categoría y Producto).

Para entender el concepto

TypeScript es como un **corrector ortográfico**: revisa tu código mientras escribes y al compilar, pero cuando el programa se ejecuta en el navegador ya no está. Al compilar, todos los tipos se **borran** y queda JavaScript puro.

`interface` y `type` sirven para describir la forma de los datos. `interface` describe objetos y se puede extender; `type` es más general: además de objetos puede expresar uniones (`'A' | 'B'`), tipos derivados de otros y combinaciones. Ninguno de los dos crea objetos ni valida nada en tiempo de ejecución.

Qué responder

- **TypeScript verifica en desarrollo y compilación.** Sus tipos no existen en runtime: no hay ningún objeto `Producto` que valide el JSON que llega del backend.
- Uso **`interface`** para formas de objetos: `Producto`, `Categoria`, `ProductoFormData`, `ProductoCreateRequest`. `ProductoUpdateRequest` **extiende** la de creación y agrega `activo`.
- Uso **`type`** cuando necesito algo que una interface no expresa bien: la unión de unidades de medida derivada de una constante (`type UnidadMedida = (typeof UNIDADES_MEDIDA)[number]`) o el tipo de errores del formulario (`Partial<Record<keyof ProductoFormData, string>>`).
- Hay **tres tipos** por entidad, cada uno con un propósito: el modelo de la respuesta (`Producto`), el estado del formulario (`ProductoFormData`, todo texto) y el payload que se envía (`ProductoCreateRequest`/`UpdateRequest`, ya con números).
- El proyecto usa TypeScript 6, que trae `strict` activado por defecto: por eso `descripcion: string | null` obliga a tratar el `null` antes de usarlo.
- Ni `interface` ni `type` "crean" objetos: los objetos se crean con literales `{...}` o llegan del `JSON.parse` de la respuesta.

Muéstralo en el código

- `src/features/productos/models/Producto.ts:2-3` (`as const` + `type UnidadMedida`) y `:19-29` (`interface Producto`).
- `src/features/productos/types/ProductoFormData.ts:7-16` y `types/ProductoRequests.ts:4-17` (`extends`).
- `src/features/productos/utils/productoValidation.ts:4` (`type ProductoFormErrors`).
- `src/api/apiClient.ts:67`: `(await response.json()) as T` — el `as` sólo "confía", no comprueba.

Si el backend envía un JSON con un campo faltante, ¿TypeScript lo bloquea automáticamente en runtime? ¿Por qué?

No. Los tipos se borran al compilar y `apiFetch` hace `response.json() as T`, que es una afirmación, no una validación. Si falta `nombre`, el objeto llega igual y en pantalla aparecerá `undefined` o se romperá un `.toLowerCase()`. Para validar en runtime haría falta código explícito (comprobaciones manuales o una librería de esquemas como Zod). Por eso el contrato con el backend debe mantenerse sincronizado y probarse.

Pregunta 05 · CRÍTICA

¿Por qué el frontend no debe copiar ciegamente las @Entity JPA del backend? Defiende tus contratos TypeScript.

Para entender el concepto

Hay tres "versiones" de un mismo dato y cada una sirve a un público distinto:

- **Modelo de persistencia** (entidad JPA): cómo se guarda en la base. Tiene detalles internos (fechas de auditoría, relaciones como objetos completos).
- **Contrato HTTP** (DTO): lo que la API decide mostrar al mundo. Es el acuerdo entre backend y frontend.
- **Modelo de UI** (FormData): cómo la pantalla necesita el dato mientras el usuario lo edita (por ejemplo, números como texto).

Copiar la entidad sería como que un cliente de un banco recibiera el formato interno de la base de datos del banco: cualquier cambio interno lo rompería, y vería cosas que no debería.

Qué responder

- El frontend **no conoce** las entidades JPA; conoce el **contrato HTTP**, es decir, los DTO de respuesta. `Producto.ts` refleja `ProductoResponse`, no `ProductoJpaEntity`.
- Diferencias concretas: `ProductoJpaEntity` tiene `CategoriaJpaEntity categoria` (el objeto completo, LAZY), `createdAt` y `updatedAt`; `CategoriaJpaEntity` tiene la lista `productos`. La API expone sólo `categoriaId` y no expone las fechas. Copiar la entidad generaría campos que nunca llegan, ciclos categoría ↔ productos y datos internos.
- `ProductoFormData` difiere de `Producto` y del DTO Java porque modela la **edición**: `categoriaId`, `stockMinimoDefault` y `precioReferencial` son `string` (los inputs entregan texto) y no tiene `id`. Recién después de validar se arma `ProductoCreateRequest` con números.
- **Riesgos de copiar la entidad:** acoplamiento (un cambio interno de la base rompe el frontend), exposición de campos internos y dependencia de detalles que el backend puede cambiar sin avisar.

- Los contratos TypeScript están en un solo lugar por módulo (`models/` y `types/`), así que si el contrato cambia, el cambio se concentra ahí y en el service.

Muéstralo en el código

- Frontend: `features/productos/models/Producto.ts:15-29` (comentario: "Contrato de ProductoResponse del backend") y `features/categorias/models/Categoria.ts:1-11` ("No es la entidad JPA... sin createdAt ni updatedAt").
- Backend para comparar: `intellij/stockflow-backend/.../product/infrastructure/adapter/out/persistence/entity/ProductoJpaEntity.java:33-65` vs `.../adapter/in/web/dto/ProductoResponse.java:7-12`.
- `features/productos/types/ProductoFormData.ts` y `types/ProductoRequests.ts`.

Si el backend renombra idCliente a id pero el frontend tiene un adapter, ¿dónde debería concentrarse el cambio?

En el adapter o mapper de ese recurso: la función que traduce la respuesta del backend al modelo del frontend. El resto (páginas, tablas, formularios) sigue usando el mismo modelo y no se entera. En StockFlow el modelo coincide 1 a 1 con el DTO, así que el "adapter" es el par `models/` + `services/` del módulo: el cambio se haría en `Producto.ts` y, si hiciera falta traducir nombres, en `productoService.ts` antes de devolver los datos. Nunca habría que tocar cada componente.

Pregunta 06 · CRÍTICA

> *Explica el recorrido exacto de una URL como /clientes (en StockFlow, /productos) dentro de React Router.*

Para entender el concepto

En una SPA el servidor entrega siempre el mismo HTML. El que decide qué pantalla mostrar según la URL es el **router**, dentro del navegador. Es como un recepcionista que mira el número de oficina (la URL) y te lleva a la puerta correcta, sin salir del edificio.

Un **layout** es un marco fijo (menú, encabezado) con un hueco en el medio: ese hueco es el `<Outlet>`, donde el router coloca la página de la URL actual.

Qué responder

1. `BrowserRouter` (en `main.tsx`) lee la URL del navegador con la History API y avisa cuando cambia.
2. `<Routes>` compara esa URL con sus `<Route>` y elige la que coincide.
3. La ruta de layout `<Route element={<MainLayout />}>` no tiene `path`: envuelve a todas las rutas hijas. Se renderiza `MainLayout` (Sidebar + Header).
4. La ruta hija `path="productos"` coincide: su elemento `<ProductosPage />` se pinta dentro del `<Outlet />` de `MainLayout`. Como la página es `lazy`, la primera vez se descarga su código mientras se muestra el `Suspense` con un esqueleto.
5. Si nada coincide, `path="*"` muestra `NotFoundPage`.

- **Route decide qué elemento renderizar según la URL**; no hace peticiones al backend.
- **Ruta del navegador ≠ endpoint REST:** `/productos` es una dirección de la interfaz (qué pantalla ver); `http://localhost:8080/api/productos` es un recurso del backend (qué datos pedir). La pantalla `/productos` es la que, una vez montada, pide los datos a `/api/productos`.

Muéstralo en el código

- `src/main.tsx:18` (`<BrowserRouter>`).
- `src/routes/AppRouter.tsx:13` (`lazy` de ProductosPage), `:39` (ruta de layout), `:43` (`path="productos"`), `:51` (`path="*"`).
- `src/layouts/MainLayout.tsx:83-85` (`<Suspense>` + `<Outlet />`).
- El comentario de `AppRouter.tsx:21-27`: "/categorias es una ruta del NAVEGADOR, no el endpoint /api/categorias".

¿Qué diferencia hay entre /clientes del frontend y http://localhost:8080/clientes del backend aunque se escriban parecido?

`/productos` en el puerto 5173 lo resuelve **React Router en el navegador** y devuelve una pantalla (componentes). `http://localhost:8080/api/productos` lo resuelve **Spring Boot en el servidor** y devuelve datos JSON. Viven en orígenes distintos, tienen responsabilidades distintas y pueden cambiar de forma independiente: podría renombrar la pantalla a `/catalogo` sin tocar el backend. En StockFlow además el backend usa el prefijo `/api`, lo que ayuda a no confundirlos.

Pregunta 07

Diferencia Link, NavLink, navegación declarativa y estado activo del menú.

Para entender el concepto

Un `<a href>` normal le pide al servidor una página nueva: se recarga todo y se pierde lo que había en memoria. `Link` parece un enlace, pero intercepta el clic y sólo cambia la URL; React Router cambia la pantalla sin recargar. `NavLink` es un `Link` que además sabe si su ruta es la actual, ideal para resaltar la opción activa del menú.

Declarativo significa que describes *a dónde* lleva el enlace (`to="/productos"`) en el JSX, en lugar de escribir instrucciones de *cómo* navegar.

Qué responder

- `Link` y `NavLink` evitan la **recarga completa** que haría un `<a>` tradicional: usan la History API, React Router renderiza la nueva ruta y el estado de la app (tema, sesión, toasts) se conserva.
- `NavLink` recibe `isActive` en su `className` (o `style`) y así el menú resalta la sección actual. Para `/` se usa `end`, porque si no "Inicio" estaría activo en todas las rutas (todas empiezan con `/`).
- **Navegación declarativa:** el menú se genera con `map` sobre `NAV_SECTIONS` (una sola fuente de verdad que también usan los Breadcrumbs y la búsqueda del Header). La navegación **imperativa** también existe (`useNavigate`) y la uso cuando la navegación es consecuencia de una acción, por ejemplo "Ver productos" desde el menú de una categoría.

Muéstralo en el código

- `src/components/common/Sidebar.tsx:52-57`: `<NavLink to={item.to} end={item.to === '/'} className={({ isActive }) => ...sidebar__link--active...}>`.
- `src/app/navigation.ts:38-62`: `NAV_SECTIONS`.
- `Link` simple: `Sidebar.tsx:32` (logo → Inicio) y `features/categorias/components/CategoriaTable.tsx:50` (`/productos?categoria=ID`).
- Imperativa: `CategoriaTable.tsx:71` (`navigate(`/productos?categoria=${c.id}`)`).

Demuéstralo

Con Network abierto, navega entre Categorías y Productos desde el menú: no aparece una nueva petición del documento HTML (sólo las de datos `/api/...`), y la opción activa cambia de color.

Si usas window.location para cada opción del menú, ¿qué comportamiento de SPA pierdes?

Cada clic sería una **recarga completa**: el navegador vuelve a pedir `index.html`, se descarga y ejecuta de nuevo el JavaScript, React se monta desde cero y se pierde todo el estado en memoria (datos cargados, filtros, toasts, sesión demo si no estuviera persistida). Además la transición es más lenta, parpadea la pantalla y no funcionaría el resaltado de `NavLink` ni el `Suspense` de las páginas lazy.

Pregunta 08

Diferencia Layout, Page, Feature y componente UI reutilizable usando archivos reales.

Para entender el concepto

Piensa en una revista: el **layout** es el diseño fijo de cada página (encabezado, márgenes, número de página); la **page** es un artículo concreto; la **feature** es una sección temática completa (deportes, cultura) con todo lo que necesita; y los **componentes UI** son piezas de imprenta genéricas (títulos, recuadros) que sirven en cualquier sección.

Qué responder

- **Layout**: estructura persistente que no cambia entre pantallas. `MainLayout` tiene Sidebar, Header y el `Outlet`.
- **Page**: compone una pantalla completa y coordina: `ProductosPage` une hooks de datos, filtros, tabla, modal de formulario y diálogo de confirmación, y decide qué hacer en cada acción.
- **Feature**: agrupa una capacidad de negocio con todo lo suyo: `features/productos/` tiene `models`, `types`, `services`, `hooks`, `components` (`ProductoForm`, `ProductoTable`), `pages` y `utils`.
- **UI reutilizable**: componentes genéricos sin conocimiento del negocio, configurables por props: `Button`, `Modal`, `ConfirmDialog`, `Pagination`, `SearchInput`, `StatusBadge`, `DataTable`, en `components/ui/`.
- **Límite:** si un componente sabe qué es un producto, va en la feature; si sirve igual para cualquier entidad, va en `components/ui`. Así `components/` no se convierte en un "cajón de sastre".

Muéstralo en el código

- Layout: `src/layouts/MainLayout.tsx`.
- Page: `src/features/productos/pages/ProductosPage.tsx` y `features/categorias/pages/CategoriasPage.tsx`.
- Feature: carpeta `src/features/productos/`.
- UI: `src/components/ui/StatusBadge.tsx`, `Modal.tsx`, `Pagination.tsx`.

¿Dónde pondrías un StatusBadge genérico y dónde un VehiculoForm (en StockFlow, ProductoForm)? Justifica.

`StatusBadge` en `components/ui/`: sólo recibe un estado (activo/inactivo) y lo muestra; lo usan categorías, productos y ubicaciones. `ProductoForm` en `features/productos/components/`: conoce los campos del producto, sus validaciones (`validarProducto`), la relación con categorías y llama a `productoService`. Si lo pusiera en `components/ui`, un componente "genérico" dependería del negocio y se rompería la dirección de dependencias.

Pregunta 09

Explica import.meta.env, las variables VITE_* y por qué no debes guardar secretos ahí.

Para entender el concepto

Las **variables de entorno** permiten cambiar la configuración (por ejemplo, la URL del backend) sin tocar el código. Vite las lee de archivos `.env` y, **al compilar**, reemplaza `import.meta.env.VITE_ALGO` por su valor literal dentro del JavaScript.

Ese JavaScript se descarga en el navegador de cualquier usuario, que puede abrirlo con F12. Por eso todo lo que esté en una variable `VITE_` es **público**. El prefijo es justamente un filtro de seguridad: Vite sólo expone las variables que empiezan con `VITE_`.

Qué responder

- StockFlow usa `VITE_API_URL` (el equivalente de `VITE_API_BASE_URL` del examen). Se lee en **un solo lugar**, `config/env.ts`, y si falta se lanza un error claro al arrancar en vez de fallar en cada petición.
- Su tipo está declarado en `vite-env.d.ts` (`interface ImportMetaEnv { readonly VITE_API_URL: string }`), así TypeScript la conoce.
- **Un archivo por modo:** `.env.development` (lo usa `npm run dev`: `http://localhost:8080/api`), `.env.production` (lo usa `npm run build`: `/api`, relativa, porque en producción nginx reenvía `/api/` al backend), `.env.example` (plantilla documentada) y `.env.local` para valores personales que no se suben a Git. En las pruebas, la URL se define en `vite.config.ts` (`test.env`).
- **Nunca secretos:** el valor queda incrustado en el bundle y cualquiera lo ve en DevTools → Sources. Una URL pública está bien; una contraseña o un token privado, jamás.

Muéstralo en el código

- `src/config/env.ts:2-6`: `export const API_URL = import.meta.env.VITE_API_URL` + el `throw` si falta.
- `src/vite-env.d.ts:4-6`: tipado de la variable.
- `.env.development`, `.env.production`, `.env.example` (cada uno con un comentario: "NUNCA pongas contraseñas ni tokens aquí").
- `vite.config.ts:28`: `env: { VITE_API_URL: ... }` para Vitest.

¿Por qué una password de base de datos dentro de VITE_DB_PASSWORD sería un error grave aunque esté en .env?

Porque el prefijo `VITE_` hace que Vite la copie dentro del JavaScript que se envía al navegador. Estar en un `.env` no la protege: el archivo es sólo el origen; el resultado (`dist/assets/*.js`) es público y cualquiera podría leer la contraseña y conectarse a la base. Además el frontend nunca debería hablar con la base de datos: eso es trabajo del backend, que guarda sus credenciales en variables de entorno del servidor (`DB_PASSWORD` en `application.yml`).

Pregunta 10

Explica el flujo unidireccional de datos en React: props hacia abajo y eventos/callbacks hacia arriba.

Para entender el concepto

En React los datos **bajan** de padre a hijo como **props** (parámetros de solo lectura). Si el hijo necesita que algo cambie, no lo cambia él: **avisa hacia arriba** llamando a una función que el padre le pasó (un **callback**). El padre, que es dueño del estado, decide y actualiza; React vuelve a renderizar y los hijos reciben las props nuevas.

Es como una empresa: el jefe reparte instrucciones (props); un empleado que detecta un problema no cambia el presupuesto por su cuenta, sino que avisa al jefe (callback).

Qué responder

- `ProductosPage` es dueña del estado (lista de productos, producto en edición, producto pendiente de borrar). Le pasa a `ProductoTable` las props `productos`, `categoriasPorId` y `stock` (datos hacia abajo) y los callbacks `onEdit` y `onDelete` (avisos hacia arriba).
- **Props vs estado local:** las props llegan de afuera y el componente no las modifica; el estado (`useState`) pertenece al componente y sólo él lo cambia con su setter. `ProductoForm` recibe `producto` por props pero copia sus valores a su propio estado `formData` para editarlos.
- **El hijo no muta el estado del padre:** si la tabla modificara el array `productos` directamente, React no se enteraría (no hay setter de por medio), no habría re-render y la Page quedaría con datos inconsistentes. Además la tabla dejaría de ser reutilizable.

Muéstralo en el código

- Bajada de props y callbacks: `src/features/productos/pages/ProductosPage.tsx:260-270`.
- El hijo sólo emite: `features/productos/components/ProductoTable.tsx:21-22` (props `onEdit`/`onDelete`) y `:88`, `:92` (`onSelect={() => onEdit(p)}`).
- Formulario → Page: `ProductosPage.tsx:303-311` (`onSaved`, `onCancel`, `onStale`).

Cuando ClienteTable (ProductoTable) hace clic en Editar, ¿cómo llega esa intención hasta ClientesPage (ProductosPage) sin que la tabla haga un PUT?

La tabla llama a `onEdit(p)`, que es la función `handleEdit` que le pasó la Page. `handleEdit` (`ProductosPage.tsx:110-120`) pide el producto actualizado con `GET /productos/{id}`, guarda el resultado en `editing` y abre el modal con `ProductoForm`. La tabla no sabe qué pasó después: sólo comunicó "el usuario quiere editar este producto". El PUT ocurre recién cuando el usuario guarda el formulario.

Bloque B — Componentes, estado, formularios y UI

Objetivo del bloque: verificar que dominas el modelo mental de React: props, estado, formularios controlados y derivación de la interfaz.

Pregunta 11 · CRÍTICA

> *Explica un input controlado con useState, value y onChange, paso a paso.*

Para entender el concepto

Un input HTML guarda su propio texto. En un **input controlado**, en cambio, la "verdad" del valor vive en el **estado de React** y el input sólo muestra lo que ese estado dice (`value`). Cuando el usuario teclea, el input no se cambia solo: avisa con `onChange`, nosotros actualizamos el estado y React vuelve a dibujar el input con el valor nuevo.

Es como un marcador de un partido manejado por un árbitro: el público (usuario) grita "¡gol!", pero el número sólo cambia cuando el árbitro (setState) lo anota.

Qué responder

1. `ProductoForm` guarda todos los campos en un objeto de estado: `const [formData, setFormData] = useState<ProductoFormData>(...)`.
2. Cada input recibe `value={formData.codigo}` y `onChange={handleChange}`, y tiene un `name` igual a la propiedad del estado.
3. El usuario escribe una letra → el navegador dispara el evento → React llama a `handleChange(e)`.
4. `handleChange` lee `e.target.name` y `e.target.value` (o `checked` si es un switch) y llama a `setFormData(prev => ({ ...prev, [name]: valor }))`: crea un **objeto nuevo** copiando el anterior y cambiando un solo campo (no muta el estado). Además borra el error de ese campo.
5. React agenda un **nuevo render**; en ese render `formData.codigo` ya tiene la letra nueva y el input la muestra a través de `value`.

- **Estado de React vs valor del DOM:** el estado es la fuente de verdad; el DOM sólo refleja lo que el estado dice. Gracias a eso puedo validar, transformar o reiniciar el formulario cambiando el estado.
- Un solo `handleChange` sirve para todos los campos gracias a `[name]` (propiedad calculada).

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:72` (`useState<ProductoFormData>`).
- `ProductoForm.tsx:90-95` (`handleChange`).
- `ProductoForm.tsx:156`: `<Input name="codigo" value={formData.codigo} onChange={handleChange} />`.

Demuéstralo

Abre "Nuevo producto", escribe en Código y muestra con React DevTools (pestaña Components → `ProductoForm` → hooks) cómo cambia `formData.codigo` letra por letra.

¿Qué ocurre si pones value={nombre} pero eliminas onChange y no cambias el estado por otra vía?

El input queda **de solo lectura**: el usuario teclea, pero en el siguiente render React vuelve a poner `value={nombre}`, que no cambió, y la letra "desaparece". React además muestra en consola el aviso "You provided a `value` prop to a form field without an `onChange` handler". Para que eso sea intencional se usa `readOnly`; si se quiere un input no controlado se usa `defaultValue`.

Pregunta 12

¿Por qué VehiculoFormData.anio o clienteId (en StockFlow: categoriaId, stockMinimoDefault, precioReferencial) pueden ser string aunque el dominio use number?

Para entender el concepto

Todo lo que viene de un `<input>` o un `<select>` es **texto**, aunque el usuario escriba números. Mientras edita, el campo puede estar vacío, a medio escribir ("12.") o con algo inválido ("abc"). Si lo guardáramos como número perderíamos esa información. Por eso el estado del formulario modela la **edición** (texto) y sólo al final, cuando todo es válido, se convierte al tipo del **dominio** (número).

Qué responder

- `ProductoFormData` declara `categoriaId`, `stockMinimoDefault` y `precioReferencial` como `string` porque así los entregan los controles HTML.
- Ese texto permite representar estados intermedios válidos para la edición: vacío (`''`), "sin precio" (opcional) o un valor a medio escribir.
- **Primero se valida** el texto: `validarProducto` comprueba que la categoría exista y esté activa, que el stock mínimo sea un entero ≥ 0 (`/^\d+$/`) y que el precio tenga hasta 2 decimales.
- **Después se convierte**: `aPayload` arma el `ProductoCreateRequest` con `Number(...)`, y el precio vacío pasa a `null` (no a 0). Al editar se hace el camino inverso: `aFormulario` pasa los números a texto con `String(...)`.

Muéstralo en el código

- `src/features/productos/types/ProductoFormData.ts:3-16` (con el comentario que lo justifica).
- `features/productos/utils/productoValidation.ts:22-48` (validación del texto).
- `features/productos/components/ProductoForm.tsx:53-63` (`aPayload`: "Sólo se llama DESPUÉS de validar") y `:38-51` (`aFormulario`).

¿Qué riesgo tiene hacer Number("") o Number("abc") sin validar primero?

`Number("")` devuelve **0**: un campo vacío se convertiría en un stock mínimo 0 o un precio 0 válidos, sin que el usuario lo haya elegido (un dato falso silencioso). `Number("abc")` devuelve **NaN**, que `JSON.stringify` convierte en `null`: el backend recibiría un valor nulo o fallaría con un 400 poco claro. Validar antes permite mostrar un mensaje preciso junto al campo y no enviar la petición.

Pregunta 13

> *Explica los eventos tipados del formulario y el uso de preventDefault().*

Para entender el concepto

Cuando el usuario interactúa, el navegador crea un **evento** con información (qué elemento, qué valor). React lo entrega a tu función. TypeScript permite decir de qué tipo es ese evento para que el editor sepa qué propiedades existen: `ChangeEvent<HTMLInputElement>` tiene `target.value`; `FormEvent<HTMLFormElement>` es el envío de un formulario.

Por defecto, al enviar un `<form>` el navegador **recarga la página** (o navega a la URL del atributo `action`). En una SPA eso borraría todo el estado; `preventDefault()` cancela ese comportamiento por defecto.

Qué responder

- `handleChange` recibe `ChangeEvent<HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement>`: un único handler tipado para inputs, selects y textareas. Con el tipo, TypeScript sabe que existen `e.target.name` y `e.target.value`.
- `handleSubmit` recibe `FormEvent<HTMLFormElement>`. Se conecta con `<form onSubmit={handleSubmit}>`, así **un solo punto** recibe el envío tanto por el botón Guardar (`type="submit"`) como por la tecla Enter.
- Lo primero que hace es `e.preventDefault()`: evita la recarga/navegación tradicional del formulario HTML; desde ahí el envío lo hace JavaScript con `fetch`.
- El formulario tiene `noValidate`: desactiva los globos de validación nativos del navegador para que los errores se muestren con el estilo y los textos de StockFlow (los de `validarProducto`).

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:1` (import de `ChangeEvent` y `FormEvent` como tipos).
- `ProductoForm.tsx:90` (`handleChange` tipado) y `:97-98` (`handleSubmit` + `preventDefault`).
- `ProductoForm.tsx:128` (`<form onSubmit={handleSubmit} noValidate>`) y `:204` (botón `type="submit"`).

Si no llamas preventDefault en un formulario SPA, ¿qué comportamiento inesperado puede aparecer?

El navegador envía el formulario a la URL actual y **recarga la página**: se pierden el estado de React, los datos cargados y el modal abierto; la petición `fetch` en curso se cancela y puede que nunca se guarde, o que se guarde pero el usuario no vea el resultado. En la URL puede aparecer `?codigo=...&nombre=...` con los datos del formulario.

Pregunta 14

> *Separa validación de UI, validación del backend y constraints de base de datos.*

Para entender el concepto

Hay tres "porteros" y cada uno cumple un rol distinto:

- **Validación de UI** (React): ayuda al usuario a corregir rápido, antes de enviar. Es comodidad, no seguridad: cualquiera puede saltarla con Postman o modificando el JavaScript.

- **Validación del backend**: la autoridad. Revisa formato (Bean Validation) y reglas de negocio que necesitan consultar datos (¿la categoría está activa?).
- **Constraints de la base** (UNIQUE, FK, CHECK, NOT NULL): la última línea de defensa. Garantizan la integridad incluso si dos peticiones llegan a la vez.

Qué responder

- **El frontend mejora la experiencia, no es frontera de seguridad.** El backend es la autoridad final (RF-03).
- Ejemplos en StockFlow:
  - **UI** (`validarProducto`): código obligatorio, máximo 40 caracteres, sólo letras/números/guiones; stock mínimo entero ≥ 0; precio con 2 decimales; la categoría debe estar activa.
  - **Backend**: `@Valid` en el DTO (400) y reglas en `ProductoService`: código duplicado (409), categoría inexistente (404), mover a una categoría inactiva (422).
  - **Base de datos**: `uq_producto_codigo UNIQUE`, `fk_producto_categoria`, `ck_producto_stock_minimo CHECK (≥ 0)`, `ck_producto_precio`, `ck_producto_unidad_medida`.
- **Errores del servidor aunque el formulario parezca válido:** la UI no puede saber si el código ya existe. Cuando el backend responde 409 o 400, `ProductoForm` muestra el `Alert` con su mensaje y pinta los `fieldErrors` que vinieron del servidor junto a cada campo.
- Repetir validaciones en capas es **intencional**: cada capa protege algo distinto.

Muéstralo en el código

- UI: `src/features/productos/utils/productoValidation.ts:15-50`.
- Errores del servidor en el formulario: `features/productos/components/ProductoForm.tsx:114-121` y `:130-134`.
- Base de datos: `intellij/stockflow-backend/src/main/resources/db/migration/V1__creacion_completa_stockflow.sql:103-109`.
- Prueba: `features/categorias/pages/CategoriasPage.test.tsx:105` ("muestra en el modal el conflicto 409 del backend al crear").

Si React valida una placa (un código) como única y dos usuarios envían al mismo tiempo, ¿qué capa debe resolver finalmente la carrera?

La **base de datos**, con la constraint `UNIQUE (codigo)`. Las dos validaciones previas (la de React y la consulta "¿existe?" del servicio) pueden pasar a la vez para ambos usuarios. Al insertar, PostgreSQL acepta el primero y rechaza el segundo; el backend traduce ese rechazo (`DataIntegrityViolationException`) a un **409** y el frontend del segundo usuario muestra "código repetido". Además, React ni siquiera puede saber qué códigos existen sin preguntar al servidor.

Pregunta 15 · CRÍTICA

Explica cómo un componente hijo comunica acciones al padre mediante props callback.

Para entender el concepto

Un **callback** es una función que el padre le entrega al hijo para que el hijo "llame de vuelta" cuando algo pasa. El hijo no sabe qué hará el padre: sólo avisa. A esto se le llama **inversión de**

control: el componente visual emite *intenciones* ("quieren borrar este producto"); la página decide los *efectos* (confirmar, llamar al backend, actualizar la lista).

Qué responder

- `ProductoTable` recibe `onEdit` y `onDelete`. En el menú de cada fila, "Editar" llama a `onEdit(p)` y "Eliminar" a `onDelete(p)`. Nada más.
- `ProductosPage` decide el efecto: `onEdit` es `handleEdit` (GET por id y abrir el modal); `onDelete` guarda el producto en `pendingDelete` y abre el `ConfirmDialog`.
- `ConfirmDialog` también emite intenciones: `onConfirm` y `onCancel`. Recién en `confirmDelete` la Page llama a `productoService.eliminar`.
- `ProductoForm` avisa con `onSaved(producto, modo)`, `onCancel` y `onStale` (el registro ya no existe, recargar).
- **Callback ≠ llamada HTTP directa:** el callback sólo comunica; el efecto (HTTP, estado, toast) lo decide quien tiene el contexto.

Muéstralo en el código

- Hijo: `src/features/productos/components/ProductoTable.tsx:21-22` y `:88-93`.
- Padre: `features/productos/pages/ProductosPage.tsx:265-269` (callbacks) y `:128-144` (`confirmDelete`).
- Diálogo: `src/components/ui/ConfirmDialog.tsx:16` ("No sabe QUÉ se elimina: la Page decide qué service llamar").

¿Por qué ClienteTable (ProductoTable) no debería importar clienteService solo para eliminar una fila?

Porque la tabla quedaría acoplada al backend y tendría que hacer cosas que no le corresponden: confirmar, manejar el 409, mostrar el toast y actualizar una lista que no es suya (la lista vive en la Page). No podría reutilizarse con otros datos ni probarse sin simular HTTP, y la Page no se enteraría del borrado para sincronizar su estado. Con el callback, la tabla es sólo presentación. En StockFlow `ProductoTable.tsx` no importa ningún service.

Pregunta 16

Explica map(), filter(), find() y key en el renderizado de listas.

Para entender el concepto

- `map` transforma cada elemento de un array en otra cosa (por ejemplo, un producto en una fila `<tr>`).
- `filter` devuelve un array nuevo sólo con los elementos que cumplen una condición.
- `find` devuelve el **primer** elemento que cumple la condición (o `undefined`).

Ninguno modifica el array original: devuelven algo nuevo. La **key** es una etiqueta única que React usa para reconocer cada elemento de una lista entre un render y otro, como el número de documento de una persona en una fila.

Qué responder

- **map** genera filas y opciones: `DataTable` recorre `rows.map(row => <tr key={rowKey(row)}>)` y `ProductoForm` hace `opciones.map(c => <option key={c.id}>)`.
- **filter** produce subconjuntos: búsqueda y filtros en `ProductosPage`; quitar la fila borrada (`prev.filter(p => p.id !== producto.id)`).
- **find** localiza un elemento: `validarProducto` busca la categoría elegida con `categorias.find(c => c.id === categoriaId)`.
- Los tres son **inmutables**: devuelven un array o valor nuevo; por eso funcionan bien con el estado de React, que necesita referencias nuevas para detectar cambios.
- **key estable:** uso el `id` persistente del backend (`rowKey={(p) => p.id}`). Así React sabe qué fila es cuál aunque se borren, agreguen o reordenen.

Muéstralo en el código

- `src/components/ui/DataTable.tsx:51` (`<tr key={rowKey(row)}>`) y `features/productos/components/ProductoTable.tsx:83` (`rowKey={(p) => p.id}`).
- `features/productos/components/ProductoForm.tsx:146-147` (`map` + `key`).
- `features/productos/pages/ProductosPage.tsx:80-89` (`filter`) y `:134` (filter al borrar).
- `features/productos/utils/productoValidation.ts:23` (`find`).

¿Qué problema aparece al usar el índice como key si eliminas o reordenas filas?

El índice no identifica al elemento sino su **posición**. Si borro la fila 0, la que era 1 pasa a tener key 0 y React cree que es "la misma fila de antes" con otros datos: reutiliza su DOM y su estado interno. Resultado: un menú abierto, un checkbox marcado o un input con foco quedan en la fila equivocada, y hay re-renders innecesarios. Con el `id` eso no ocurre.

Pregunta 17

Diferencia estado fuente y estado derivado. ¿Qué datos NO conviene duplicar en useState?

Para entender el concepto

El **estado fuente** es la información mínima que no se puede calcular a partir de otra: los datos que llegaron del backend y lo que eligió el usuario (texto buscado, filtro, página). El **estado derivado** es todo lo que se calcula a partir de la fuente: la lista filtrada, la página actual de la tabla, los totales.

Si guardas lo derivado en otro `useState`, tienes dos copias que pueden dejar de coincidir. Es como anotar tu saldo bancario en un cuaderno además del banco: tarde o temprano no coinciden.

Qué responder

- **Fuente** en `ProductosPage`: `productos.data` (del backend), `search`, `status`, `page`, `pageSize` y el filtro de categoría (que vive en la URL: `?categoria=ID`).
- **Derivado**, calculado en cada render con `const`: `porTextoYCategoria`, `activos`, `filtrados`, `pagina` (con `totalItems` y `totalPaginas`) y `hayFiltros`. Ninguno está en `useState`.
- Lo mismo en el **Dashboard**: los indicadores se calculan desde los datos (`productos.filter(p => p.activo).length`), no se guardan como contadores.

- **No conviene duplicar**: listas filtradas o paginadas, totales, "¿hay resultados?", el nombre de la categoría dentro de cada producto. Excepción razonable: `useMemo` para un cálculo costoso (el `Map` de categorías), que sigue siendo derivado.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:55-59` (estado fuente) y `:79-91` (derivado).
- `src/pages/DashboardPage.tsx:25-32` (métricas derivadas).

Si cambia clientes (productos) pero olvidas actualizar clientesFiltrados guardado en otro estado, ¿qué bug visual aparece?

La tabla sigue mostrando la lista vieja: el producto recién creado no aparece, el borrado sigue visible o una edición no se refleja, hasta que el usuario toque el buscador. Los contadores (Todos/Activos/Inactivos) tampoco coincidirían con la tabla. En StockFlow no ocurre porque la lista filtrada se recalcula en cada render a partir de `productos.data`.

Pregunta 18

Diseña el render de una pantalla con loading, error, empty y success sin mezclar estados.

Para entender el concepto

Una pantalla que pide datos pasa por cuatro situaciones: **cargando** (todavía no hay respuesta), **error** (la petición falló), **vacío** (la respuesta llegó bien pero no hay registros) y **éxito** (hay datos para mostrar). Cada una necesita un mensaje distinto y no deberían verse dos contradictorias a la vez (por ejemplo, "cargando" y "no hay datos").

Qué responder

- El hook entrega `loading`, `error` y `data` (`useAsyncList`), y la Page decide con una cadena de condiciones excluyentes:
  1. **Cargando** → `TableSkeleton` (esqueleto de tabla con `aria` "Cargando productos").
  2. **Sin resultados con filtros** → `EmptyState` "Ningún producto coincide" + botón "Limpiar filtros".
  3. **Vacío real** (sin filtros y sin error) → `EmptyState` "Todavía no hay productos" + "Nuevo producto".
  4. **Éxito** → `ProductoTable` con paginación.
- **Error** → `Alert` rojo con el mensaje y un botón **Reintentar** (que llama a `reload()`). Cuando hay error no se muestra el vacío "Todavía no hay productos", para no mentir.
- Si sólo falla un dato secundario (el stock), la página sigue funcionando y muestra un `Alert` amarillo "El stock no está disponible" en vez de inventar cifras.
- **Feedback en operaciones:** los botones muestran "Guardando..." o "Eliminando..." y se deshabilitan mientras esperan; al terminar aparece un toast de éxito o error.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:164-189` (Alerts de error) y `:231-293` (loading → empty → tabla).
- `src/shared/hooks/useAsyncList.ts:14-16` (los tres estados).
- Prueba: `ProductosPage.test.tsx:48` ("si el stock no carga, la página funciona y lo avisa sin inventar cifras").

Demuéstralo

Con el frontend abierto, detén el backend y recarga Productos: aparece el Alert rojo con "Reintentar". Vuelve a levantar el backend y pulsa Reintentar: aparece la tabla. Busca un texto inexistente: aparece "Ningún producto coincide".

¿Una lista vacía [] es necesariamente un error? ¿Cómo la distinguirías de un GET que falló?

No. `[]` con status 200 es una respuesta **válida**: el backend funciona y no hay registros (o los filtros no coinciden). Un GET que falla no produce `[]`: `apiFetch` lanza un `ApiError` (status ≥ 400) o un `TypeError` (sin conexión), y `useAsyncList` lo guarda en `error`. Por eso la Page mira primero `error`: si hay error muestra el Alert; si no hay error y la lista está vacía, muestra el estado vacío.

Pregunta 19

Explica el orden correcto: datos → búsqueda/filtros → paginación → tabla.

Para entender el concepto

Primero se decide **qué** se puede ver (el "universo visible": lo que coincide con la búsqueda y los filtros) y recién después se corta en páginas. Es como buscar en una biblioteca: primero eliges todos los libros de historia, y luego los repartes en estantes de 10. Si repartes primero y buscas después, sólo buscarías dentro del primer estante.

Qué responder

1. **Datos**: `productos.data` completo del backend.
2. **Búsqueda y filtros**: texto (sin distinguir tildes ni mayúsculas, con `normalizar`), categoría y estado → `filtrados`.
3. **Paginación**: `paginar(filtrados, page, pageSize)` calcula `totalItems` (cuántos quedaron), `totalPaginas = ceil(totalItems / pageSize)` y corta con `slice(inicio, inicio + pageSize)`.
4. **Tabla**: recibe sólo `pagina.items`.

- **Reajuste de página:** al cambiar búsqueda, estado, categoría o tamaño se vuelve a la página 1 (`setPage(1)`). Además `paginar` limita la página al rango válido (`Math.min(Math.max(1, pagina), totalPaginas)`) por si un filtro o un DELETE dejaron la página actual fuera de rango.
- Una lista vacía tiene 1 página (no 0), para que el paginador no muestre "página 1 de 0".

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:79-90` (orden completo) y `:198-201`, `:219-222`, `:285-288` (`setPage(1)`).
- `src/shared/utils/paginar.ts:9-20`.
- Pruebas: `shared/utils/paginar.test.ts:14` ("ajusta una página fuera de rango a la última válida").

¿Qué error funcional aparece si paginas primero y luego buscas?

La búsqueda sólo miraría los 10 elementos de la página actual: un producto que existe pero está en la página 3 "no se encontraría", el total de resultados sería falso y el paginador mostraría páginas que, al filtrar, quedan vacías. El usuario creería que el producto no existe.

Pregunta 20

> *Explica Modal/ConfirmDialog y por qué reemplazar window.confirm mejora el diseño.*

Para entender el concepto

`window.confirm()` abre una ventanita del navegador que **congela** toda la página, no se puede diseñar, no admite textos largos ni estados como "Eliminando..." y se ve distinta en cada navegador. Un **modal** propio es un componente de React: se abre o cierra según el estado, muestra el contenido que quieras y avisa con callbacks qué eligió el usuario.

El elemento HTML `<dialog>` con `showModal()` ya resuelve gran parte de la accesibilidad: bloquea el fondo, mantiene el foco dentro y se cierra con Escape.

Qué responder

- `Modal` recibe `open`, `title`, `description`, `children`, `footer` y `onClose`. Un `useEffect` sincroniza la prop `open` con el `<dialog>` nativo: `showModal()` al abrir y `close()` al cerrar.
- **Accesibilidad**: `showModal()` lo pone encima de todo, hace inerte el fondo, atrapa el foco y al cerrar lo devuelve al botón que lo abrió; Escape dispara "cancel". El título y la descripción se enlazan con `aria-labelledby` y `aria-describedby` (ids únicos con `useId`).
- `ConfirmDialog` es un `Modal` pequeño con mensaje, "Cancelar" y "Eliminar". Recibe `confirming` para mostrar "Eliminando..." y bloquear el cierre mientras la petición está en curso.
- **El modal no ejecuta reglas de negocio:** no sabe qué se borra ni llama a services. Sólo emite `onConfirm`/`onCancel`; la Page decide. Incluso adapta el mensaje: si el producto tiene stock, avisa que el sistema no permitirá borrarlo y sugiere desactivarlo.

Muéstralo en el código

- `src/components/ui/Modal.tsx:21-37` (comentario + `showModal`/`close`) y `:40-44` (atributos aria).
- `src/components/ui/ConfirmDialog.tsx:16-52`.
- Uso: `features/productos/pages/ProductosPage.tsx:314-327`.
- Pruebas: `components/ui/Modal.test.tsx:7` y `:22`.

Si el usuario cierra el modal de DELETE, ¿qué estados deben quedar intactos y qué request NO debe ocurrir?

Al cancelar sólo se ejecuta `setConfirmOpen(false)`. **No debe salir ninguna petición DELETE** (en Network no aparece nada), la lista `productos.data` queda igual, la fila sigue en la tabla, y se conservan la página, la búsqueda y los filtros. `deleting` sigue en `false`. El `pendingDelete` puede quedar guardado sin efecto: se sobrescribe la próxima vez que se pida borrar.

Bloque C — HTTP, services y CRUD fullstack

Objetivo del bloque: recorrer solicitudes reales entre React y el backend Spring Boot y diagnosticar fallos.

Pregunta 21 · CRÍTICA

> *Explica el flujo completo de un GET desde React hasta Spring Boot y de regreso a la pantalla.*

Para entender el concepto

Un GET es una pregunta: "dame la lista de productos". El navegador la envía por HTTP, el servidor responde con un **status** (200, 404…) y un **body** en JSON (texto). JavaScript convierte ese texto en objetos, React los guarda en el estado y, como el estado cambió, vuelve a dibujar la pantalla.

Hay tres "dueños" en el camino: **tu código** (Page, hook, service, apiClient), la **Web API del navegador** (`fetch`, que hace la conexión de red) y el **backend externo** (Spring Boot + PostgreSQL).

Qué responder

1. **Page**: `ProductosPage` se monta y llama a `useProductos()`.
2. **Hook**: `useProductos` usa `useAsyncList(productoService.listar)`. Su `useEffect` crea un `AbortController`, pone `loading = true` y llama al loader con la señal.
3. **Service**: `productoService.listar(signal)` llama a `apiFetch<Producto[]>('/productos', { signal })`.
4. **apiClient**: arma la URL `API_URL + '/productos'` = `http://localhost:8080/api/productos`, agrega `Accept: application/json` y llama a `fetch` (Web API del navegador).
5. **Red y backend**: el navegador envía `GET /api/productos` con el header `Origin: http://localhost:5173`; Spring aplica CORS, el controller pide los datos al caso de uso, PostgreSQL responde y Spring devuelve **200** con un JSON y el header `Access-Control-Allow-Origin`.
6. **Vuelta**: `fetch` resuelve con un `Response`; `apiFetch` revisa `response.ok` y hace `await response.json()` → array de objetos JavaScript.
7. **Estado y render**: `useAsyncList` llama a `setData(productos)` y luego `setLoading(false)`; React re-renderiza `ProductosPage`, que deriva filtros y página y pinta `ProductoTable`.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:42` → `features/productos/hooks/useProductos.ts:5-7` → `src/shared/hooks/useAsyncList.ts:21-38` → `features/productos/services/productoService.ts:7-9` → `src/api/apiClient.ts:52-68`.
- Backend: `ProductoController` (`@GetMapping` en `/api/productos`).

Demuéstralo

F12 → Network → filtro Fetch/XHR → entra a Productos. Abre la petición `productos`: **Headers** (URL, método GET, status 200, `Origin`, `Access-Control-Allow-Origin`), **Response** (el JSON) y **Initiator** (la cadena de archivos que la originó: `apiClient.ts` ← `useAsyncList.ts`).

¿En qué punto exacto se convierte la respuesta HTTP en un objeto JavaScript y en qué punto React decide volver a renderizar?

La conversión ocurre en `await response.json()` (`apiClient.ts:67`): lee el body como texto y lo parsea a objetos. El re-render se decide cuando se llama al setter de estado: `setData(resultado)` en `useAsyncList.ts:29`, justo después de `await loader(...)` (`:27`). React compara el valor nuevo con el anterior (es un array nuevo), agenda el render de `ProductosPage` y, como `setLoading(false)` ocurre en el mismo ciclo, los agrupa en un solo render (batching).

Pregunta 22

¿Por qué fetch() no considera automáticamente un 404 o 500 como rechazo de la Promise?

Para entender el concepto

Para `fetch`, "funcionó" significa **"recibí una respuesta del servidor"**, sin importar si es buena o mala. Un 404 o un 500 son respuestas completas y válidas desde el punto de vista de la red. `fetch` sólo rechaza la Promise cuando **no hubo respuesta**: sin conexión, servidor apagado, CORS bloqueado, URL imposible o petición abortada.

Por eso siempre hay que mirar `response.ok` (true si el status está entre 200 y 299) o `response.status`.

Qué responder

- `fetch` resuelve la Promise con cualquier status HTTP. Rechaza sólo ante **error de red** (lanza `TypeError: Failed to fetch`) o **aborto** (`AbortError`).
- `apiFetch` revisa `if (!response.ok)` en **un solo lugar** y, si falla, lanza un `ApiError` con el `status`, el `message` y los `fieldErrors` que envió el `GlobalExceptionHandler` del backend. Si el body no es JSON (por ejemplo, un proxy), usa el texto o el `statusText`.
- Así quedan tres casos bien diferenciados:
  - **Error HTTP** → `ApiError` con status (400, 404, 409, 422, 500).
  - **Error de red** → `TypeError` → `mensajeDeError` dice "No se pudo conectar con el backend. ¿Está corriendo StockFlowApplication en el puerto 8080?".
  - **Aborto** → `AbortError`, que se ignora (no es un error del sistema).
- Las Pages y formularios no revisan status: reciben un error ya clasificado y lo traducen con `describirError`.

Muéstralo en el código

- `src/api/apiClient.ts:47-63` (comentario "fetch NO rechaza la Promise ante un 404 o un 500" + `response.ok`).
- `apiClient.ts:21-45` (`ApiError` y `construirError`) y `:71-78` (`mensajeDeError`, caso `TypeError`).
- Pruebas: `src/api/apiClient.test.ts:25` y `:41`.

Si el servidor responde 500 con JSON, ¿entrarías siempre directamente al catch sin revisar response.ok?

No. Con un 500, `fetch` **resuelve** igual que con un 200; si no reviso `response.ok`, el código seguiría al `response.json()` y trataría el JSON de error como si fueran datos (por ejemplo, intentaría mostrar `{ status: 500, message: ... }` como lista de productos). Al `catch` sólo se llega porque `apiFetch` lanza explícitamente el `ApiError` después de comprobar `!response.ok`.

Pregunta 23 · CRÍTICA

¿Qué es CORS y por qué Postman/Swagger pueden funcionar mientras React falla en el navegador?

Para entender el concepto

Por seguridad, el navegador no deja que una página lea respuestas de otro **origen** (otro dominio, puerto o protocolo) salvo que ese otro servidor diga explícitamente "este origen tiene permiso". Esa regla del navegador se llama **CORS** (Cross-Origin Resource Sharing).

Origen = esquema + host + puerto. `http://localhost:5173` y `http://localhost:8080` comparten esquema y host, pero no puerto: son orígenes distintos. Postman no es un navegador y no aplica CORS. Swagger UI se sirve desde el mismo backend (puerto 8080), así que sus peticiones son del mismo origen.

Qué responder

- CORS es una **política del navegador**, no del backend ni de la base. El servidor sólo declara a quién permite; quien bloquea es el navegador.
- React corre en `http://localhost:5173` y la API en `http://localhost:8080`: orígenes distintos. El navegador envía el header `Origin` y sólo entrega la respuesta a JavaScript si vuelve `Access-Control-Allow-Origin` con ese origen.
- En POST, PUT y DELETE con `Content-Type: application/json`, el navegador manda antes una petición **preflight** (`OPTIONS`) preguntando si el método y los headers están permitidos.
- En StockFlow, `CorsConfig` permite en `/api/**` los orígenes de `stockflow.cors.allowed-origins` (por defecto `http://localhost:5173`, configurable con `CORS_ALLOWED_ORIGINS`), los métodos GET, POST, PUT, DELETE y OPTIONS, los headers `Content-Type` y `Accept`, y expone `Location`. Por eso Vite tiene `strictPort: true`: si arrancara en 5174, CORS fallaría.
- **Cómo se diagnostica:** la consola muestra "blocked by CORS policy"; en Network la petición aparece en rojo o con "CORS error", a veces con un `OPTIONS` previo fallido; y en JavaScript llega como `TypeError` (por eso el mensaje "No se pudo conectar con el backend").

Muéstralo en el código

- Backend: `intellij/stockflow-backend/src/main/java/com/stockflow/shared/web/CorsConfig.java:23-29` y `src/main/resources/application.yml` (`stockflow.cors.allowed-origins`).
- Frontend: `vite.config.ts:7-12` (puerto fijo) y `src/api/apiClient.ts:73-75`.
- Postman: carpeta `06 CORS` de la colección "StockFlow - Pruebas completas" (preflight permitido 200 y origen no permitido 403).

Si npm run backend:check funciona pero el navegador bloquea la llamada, ¿cuál es tu primera hipótesis y cómo la confirmas?

(StockFlow no tiene un script `backend:check`; el equivalente es `curl http://localhost:8080/api/health` o `healthService.comprobar()`, que usa la pantalla de acceso.) Si el backend responde fuera del navegador pero el navegador bloquea, la primera hipótesis es **CORS**: el origen del frontend no está permitido (por ejemplo, Vite en otro puerto o un dominio de producción no configurado). Lo confirmo en la Console ("blocked by CORS policy") y en Network: la respuesta (o el preflight `OPTIONS`) no trae `Access-Control-Allow-Origin` con mi origen. Lo corrijo en `CORS_ALLOWED_ORIGINS` del backend, no desactivando la seguridad.

Pregunta 24

Defiende la separación entre apiClient, services, hooks y componentes.

Para entender el concepto

Cada capa responde una pregunta distinta:

- **apiClient**: *¿cómo* se habla HTTP? (URL base, headers, verificar status, convertir errores).
- **service**: *¿qué* operaciones existen sobre un recurso? (listar, crear, actualizar, eliminar productos).
- **hook**: *¿cuándo* se pide y en qué estado está la petición? (cargando, error, datos, recargar, cancelar).
- **componente**: *¿qué* se ve y qué hace el usuario?

Qué responder

- `api/apiClient.ts` (`apiFetch`): mecánica HTTP común. Es la única puerta al backend.
- `features/productos/services/productoService.ts`: operaciones del recurso con nombres de negocio. Es el único lugar que conoce las rutas `/productos`; los componentes no construyen URLs.
- `shared/hooks/useAsyncList.ts` (genérico) y `features/productos/hooks/useProductos.ts` (de dominio): coordinan estado y asincronía con cancelación.
- `ProductosPage`, `ProductoTable`, `ProductoForm`: renderizan e interactúan.
- **Ventajas:** un cambio de URL base, de header o de formato de error se hace en un solo archivo; las pruebas de páginas simulan el service completo con `vi.mock` sin tocar `fetch`; y `apiFetch` se prueba aparte simulando `fetch`.

Muéstralo en el código

- `src/api/apiClient.ts:47-68` · `features/productos/services/productoService.ts:5-24` · `src/shared/hooks/useAsyncList.ts:7-40` · `features/productos/hooks/useProductos.ts`.
- Pruebas por capa: `src/api/apiClient.test.ts` (`vi.stubGlobal('fetch', ...)`) y `features/productos/pages/ProductosPage.test.tsx:14-16` (`vi.mock` de los services).

Si cambias fetch por Axios, ¿qué capas deberían cambiar idealmente y cuáles no?

Idealmente sólo `api/apiClient.ts`: `apiFetch` pasaría a usar `axios.request`, y la conversión de errores se adaptaría (Axios sí rechaza ante 4xx/5xx). Los services, hooks, páginas y componentes no cambian porque sólo dependen de la firma `apiFetch<T>(endpoint, options)`. También habría que ajustar `apiClient.test.ts`, que simula `fetch`.

Pregunta 25 · CRÍTICA

> *POST Cliente (en StockFlow, POST Producto): explica línea por línea desde Guardar hasta que la nueva fila aparece en la tabla.*

Para entender el concepto

Crear algo en la pantalla no es lo mismo que **persistirlo**. Si sólo agregara un objeto al array de React, desaparecería al recargar y nadie más lo vería. El registro existe de verdad recién cuando el backend lo guarda en PostgreSQL y devuelve **201 Created** con el objeto creado (incluido el `id` que generó la base).

Qué responder

1. **Clic en "Crear producto"**: es un `type="submit"`; el form dispara `onSubmit` → `handleSubmit`.
2. `e.preventDefault()` evita la recarga; se limpia el error anterior de la API.
3. **Validación**: `validarProducto(formData, categorias)`. Si hay errores se guardan, se enfoca el primer campo inválido y se corta con `return`: **no sale ninguna petición**.
4. **Payload**: `aPayload(formData)` convierte el texto en `ProductoCreateRequest` (`categoriaId` y números con `Number`, `trim`, precio vacío → `null`).
5. `setSaving(true)`: el botón muestra "Guardando..." y queda deshabilitado (evita doble envío).
6. **Service**: `productoService.crear(payload)` → `apiFetch('/productos', { method: 'POST', body: JSON.stringify(data) })`.
7. **apiClient**: como hay body, agrega `Content-Type: application/json` y `Accept`; `fetch` envía el POST (antes, el preflight `OPTIONS` de CORS).
8. **Backend**: `@Valid`, reglas de negocio, `INSERT` en PostgreSQL → **201 Created** con el `ProductoResponse` (ya con `id`) y el header `Location`.
9. `apiFetch` verifica `response.ok` y devuelve `response.json()` → objeto `Producto`.
10. **Estado**: el form llama a `onSaved(producto, 'create')`; la Page ejecuta `productos.setData(prev => [...prev, saved])`, cierra el modal y muestra el toast "Producto X creado".
11. **Render**: React re-renderiza la Page; la lista derivada incluye el nuevo producto y la fila aparece (si coincide con los filtros y la página actual).
12. **Errores**: si falla, el `catch` pinta los `fieldErrors` del servidor y un `Alert` con el mensaje (409 código repetido, 422 categoría inactiva, 404 categoría inexistente); el `finally` devuelve `saving` a `false`.

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:97-125` (`handleSubmit` completo), `:54-63` (`aPayload`) y `:204` (botón con `loading`).
- `features/productos/services/productoService.ts:13-15` (`crear`) y `src/api/apiClient.ts:53-59` (headers y `fetch`).
- `features/productos/pages/ProductosPage.tsx:122-126` (`handleSaved`).
- Prueba: `features/categorias/pages/CategoriasPage.test.tsx:76` ("crea una categoría desde el modal con POST y la agrega a la tabla").

Demuéstralo

Crea un producto con Network abierto: verás el `OPTIONS` (preflight) y el `POST productos` con status 201; en **Payload** el JSON enviado (con `categoriaId` numérico) y en **Response** el objeto con su `id`. Luego confirma en DataGrip: `SELECT * FROM stockflow.producto ORDER BY producto_id DESC LIMIT 1;`.

Si el backend devuelve 201 pero tu tabla no cambia, ¿qué puntos del frontend revisarías y en qué orden?

1) En Network, que el body del 201 sea el objeto esperado (con `id`). 2) Que el form llame a `onSaved` (que no haya saltado al `catch` por un error al parsear). 3) Que `handleSaved` actualice el estado con un **array nuevo** (`[...prev, saved]`) y no mutando (`prev.push` no provoca re-render). 4) Que la tabla lea de la lista derivada de ese estado y no de una copia vieja. 5) Los filtros y la paginación: el producto puede estar creado pero fuera del filtro de categoría o en otra página. 6) Que la `key` sea el `id` y no esté repetida.

Pregunta 26

GET/{id}: explica cómo seleccionas un registro para editar y por qué podrías volver a consultarlo.

Para entender el concepto

La lista que ves en la tabla es una **foto** tomada cuando cargó la pantalla. Mientras tanto, otro usuario pudo modificar o borrar ese registro. Pedir el registro por su `id` justo antes de editar garantiza que el formulario arranca con los datos **actuales del servidor**.

Qué responder

1. El usuario elige "Editar" en la fila → `ProductoTable` llama a `onEdit(p)`.
2. `handleEdit` marca `loadingDetailId = p.id` (la fila muestra que está cargando) y llama a `productoService.obtenerPorId(p.id)` → `GET /api/productos/{id}`.
3. Con la respuesta llama a `abrirFormulario(producto)`: guarda `editing`, incrementa `formKey` (React crea un formulario nuevo) y abre el modal.
4. `ProductoForm` precarga su estado con `aFormulario(producto)` (los números pasan a texto).
5. Si el GET responde **404** (alguien lo borró), se muestra el toast "El registro ya no existe" y se recarga la lista.
6. **Cancelar** cierra el modal (`setFormOpen(false)`); como la próxima apertura usa una `key` nueva, el formulario no arrastra datos de la edición anterior.

- **¿Dato local o refrescar?** El objeto de la fila sirve para mostrar; para editar, StockFlow vuelve a consultar porque la edición es un PUT de reemplazo completo y conviene partir de la versión más reciente. En una pantalla de sólo lectura bastaría el dato local.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:104-120` (`abrirFormulario` y `handleEdit`) y `:303-304` (`key={formKey}`).
- `features/productos/components/ProductoForm.tsx:38-51` (`aFormulario`) y `:72`.
- Prueba: `features/categorias/pages/CategoriasPage.test.tsx:118` ("edita con los datos actuales del servidor (GET /{id}) y guarda con PUT").

¿Qué pasa si el registro cambió en el servidor desde que cargaste la lista y editas una copia vieja?

Como el PUT reemplaza el recurso completo, enviaría los valores viejos de los campos que no toqué y **pisaría** los cambios del otro usuario sin que nadie se entere (actualización perdida). StockFlow lo reduce pidiendo el GET/{id} al abrir el formulario. Para eliminarlo del todo haría falta control de concurrencia en el backend (un número de versión con `@Version` que devuelva 409 si el registro cambió entre medio).

Pregunta 27 · CRÍTICA

PUT: explica la actualización completa y por qué el mismo formulario puede servir para Crear y Editar.

Para entender el concepto

PUT significa "reemplaza este recurso por esta versión completa". Es **idempotente**: enviar la misma petición una o diez veces deja el mismo resultado final. Crear y editar piden los mismos campos; la única diferencia es si ya existe un `id`. Por eso un solo formulario puede servir para ambos, decidiendo el modo según reciba o no un registro.

Qué responder

- **Modo**: `ProductoForm` recibe `producto: Producto | null`. `null` → crear; con objeto → editar (`isEditing = producto !== null`). Cambian el título, el texto del botón ("Crear producto" / "Guardar cambios"), el switch `activo` (sólo al editar) y el service que se llama.
- **UpdateRequest**: al editar se arma `{ ...aPayload(formData), activo: formData.activo }`, que cumple `ProductoUpdateRequest` (todos los campos, incluido `activo`, que es obligatorio en el PUT).
- **PUT**: `productoService.actualizar(id, data)` → `PUT /api/productos/{id}` → 200 con el producto actualizado.
- **Sincronizar el estado**: `handleSaved` en modo `'edit'` reemplaza el elemento con `prev.map(p => p.id === saved.id ? saved : p)`, usando la respuesta del servidor (no lo que se escribió).
- **Idempotencia**: repetir el mismo PUT deja el mismo estado final. Por eso un reintento es seguro (a diferencia de repetir un POST, que crearía un duplicado o daría 409).

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:77` (`isEditing`), `:109-112` (PUT o POST) y `:188-205` (switch y textos por modo).
- `features/productos/types/ProductoRequests.ts:14-17` (`ProductoUpdateRequest extends ...`).
- `features/productos/services/productoService.ts:16-19` y `pages/ProductosPage.tsx:123`.

Si presionas Guardar dos veces con exactamente el mismo PUT, ¿qué estado final debería quedar?

El mismo que con un solo PUT: el producto con esos valores. Esa es la idempotencia. Además, en StockFlow el segundo clic ni siquiera sale: mientras `saving` es `true`, el botón queda deshabilitado (`Button` usa `disabled={disabled || loading}`). La colección de Postman lo verifica en "02 200 Mismo PUT otra vez (idempotente)". Lo único que podría cambiar en la base es `updated_at`, que es auditoría, no estado de negocio.

Pregunta 28 · CRÍTICA

> *DELETE: explica confirmación, request, 204 No Content y sincronización de la tabla.*

Para entender el concepto

Borrar es irreversible, por eso se **confirma** antes. El backend responde **204 No Content** cuando borró con éxito: "listo, y no tengo nada que devolverte" (no hay body). La regla de oro es **no quitar la fila de la pantalla hasta que el servidor confirme**: si el backend se niega (por ejemplo porque hay datos que dependen del registro), la fila debe seguir ahí.

Qué responder

1. **Intención**: "Eliminar" en la fila → `onDelete(p)` → la Page guarda `pendingDelete` y abre el `ConfirmDialog`. Todavía no hay petición.
2. **Confirmación**: el diálogo explica la consecuencia (si el producto tiene stock, avisa que no se podrá borrar y sugiere desactivarlo). Cancelar no envía nada.
3. **Request**: al confirmar, `confirmDelete` pone `deleting = true` (sólo el botón del diálogo muestra "Eliminando..."; el resto de la pantalla sigue usable) y llama a `productoService.eliminar(id)` → `DELETE /api/productos/{id}`.
4. **204**: `apiFetch` detecta `response.status === 204` y devuelve `undefined` sin llamar a `response.json()` (que fallaría con un body vacío).
5. **Sincronizar**: **después** del `await` exitoso se quita la fila con `setData(prev => prev.filter(p => p.id !== producto.id))` y se muestra un toast.
6. **Error**: si el backend responde 409 (producto con stock o movimientos; categoría con productos), el `catch` muestra el mensaje y la fila se conserva. Si responde 404, se recarga la lista. El `finally` cierra el diálogo y vuelve `deleting` a `false`.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:266-269` (abrir confirmación), `:128-144` (`confirmDelete`) y `:314-327` (diálogo).
- `src/api/apiClient.ts:64-66` (204 sin body) y `features/productos/services/productoService.ts:20-23`.
- Pruebas: `features/categorias/pages/CategoriasPage.test.tsx:52` ("conserva la fila y muestra el mensaje del backend cuando DELETE responde 409") y `:65` ("quita la fila sólo después de que el backend confirma el DELETE").

Demuéstralo

Intenta borrar la categoría `CAT-OFI` (tiene productos): Network muestra `DELETE ... 409` y la fila sigue. Crea una categoría nueva y bórrala: `204`, la pestaña Response está vacía y la fila desaparece.

Si el backend responde 409 porque el Cliente (la Categoría) tiene Vehículos (Productos), ¿por qué sería incorrecto hacer filter() y ocultarlo igualmente?

Porque la pantalla mentiría: el registro **sigue existiendo** en la base. El usuario creería que lo borró, al recargar "reaparecería", y las demás pantallas (productos de esa categoría, dashboard) mostrarían datos inconsistentes. El 409 significa "no lo hice por una regla de integridad"; lo correcto es conservar la fila y explicar el motivo (en StockFlow: "tiene productos" o "tiene stock o movimientos").

Pregunta 29

> *Explica cómo interpreta el frontend 400, 404, 409 y 500 con ejemplos reales.*

Para entender el concepto

Los códigos 4xx dicen "el problema está en lo que **pediste**" y los 5xx "el problema está en el **servidor**". Cada uno pide una reacción distinta: un 400 se corrige editando datos; un 404 indica que el recurso ya no está; un 409 que choca con datos existentes; un 500 no lo puede arreglar el usuario. Un buen frontend traduce cada caso a un mensaje útil y **nunca finge éxito**.

Qué responder

Status	Significado	Ejemplo en StockFlow	Reacción del frontend
400	Payload inválido (Bean Validation, JSON mal formado, tipo incorrecto)	Código vacío; unidad de medida fuera del enum; stock mínimo negativo	"Revisa los datos ingresados" + `fieldErrors` pintados junto a cada campo
404	Recurso o ruta inexistente	Editar un producto que otro usuario borró; categoría inexistente	"El registro ya no existe... Se actualizó la lista" y `reload()`
409	Conflicto con datos existentes	Código `PRD-OFI-001` repetido; borrar `CAT-OFI` con productos; borrar un producto con stock	Mensaje del backend; la fila o el formulario se conservan
422	Regla de negocio (extra de StockFlow)	Mover un producto a una categoría inactiva	"La operación no cumple una regla del inventario" + detalle
500	Fallo interno	Excepción no prevista en el servidor	"El servidor tuvo un problema"; no se inventan datos
—	Sin respuesta (red/CORS)	Backend apagado	"No se pudo conectar con el backend..."

- Toda esta traducción vive en un solo archivo, `describirError`, y el mensaje del backend se muestra siempre como descripción (el backend es la fuente de verdad). Así no se convierte todo en "No hay internet".

Muéstralo en el código

- `src/api/describirError.ts:9-32` (comentario por status y función `describirError`).
- `src/api/apiClient.ts:71-78` (`mensajeDeError`).
- Uso: `features/productos/components/ProductoForm.tsx:114-121` y `pages/ProductosPage.tsx:114-117`, `:136-139`.

Dame un ejemplo (de tu proyecto) donde 409 sea más correcto que 400.

Crear un producto con el código `PRD-OFI-001`, que ya existe. La petición está **bien formada** (el código es válido: letras, números y guiones), así que no es un 400; el problema es que **choca con un dato existente** (UNIQUE de RN-06). Otro ejemplo: borrar la categoría `CAT-OFI`, que tiene productos: la petición es válida, pero la FK lo impide.

Pregunta 30

> *Después de POST/PUT/DELETE: ¿actualizas estado local o haces refetch? Compara ambas estrategias.*

Para entender el concepto

Tras modificar algo en el servidor, la lista de la pantalla queda desactualizada. Hay dos formas de ponerla al día: **actualización local** (modificar el array en memoria con lo que respondió el servidor) o **refetch** (volver a pedir toda la lista). La primera es rápida; la segunda garantiza que la pantalla coincide exactamente con el servidor.

Qué responder

- **Actualización local**: una petición menos, respuesta instantánea y la tabla no parpadea. Riesgo: si se usa lo que el usuario escribió en lugar de la respuesta, o si la respuesta es parcial, la pantalla puede quedar distinta del servidor; y se duplica lógica (saber dónde insertar, cómo ordenar).
- **Refetch**: la pantalla queda alineada con el servidor (campos calculados, cambios de otros usuarios). Cuesta otra petición y un momento de carga.
- **Estrategia de StockFlow**, distinta por operación:
  - POST y PUT → actualización local **con la respuesta del servidor** (`saved`), no con el formulario.
  - DELETE → `filter` local sólo después del 204.
  - 404 en cualquier operación → **refetch** (`reload()`), porque la lista ya no es confiable.
  - "Reintentar" y "Actualizar" (dashboard) → refetch.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:122-126` (local con `saved`), `:134` (filter tras DELETE), `:116` y `:139` (`reload()` ante 404).
- `src/shared/hooks/useAsyncList.ts:19` (`reload` cambia `reloadToken` y vuelve a correr el Effect) y `:40` (expone `setData`).

Si el backend calcula un campo que el frontend no conoce, ¿qué ventaja tiene usar la respuesta del servidor o hacer refetch?

Que el campo calculado (un `id` generado, un valor normalizado, una fecha, un total) llega correcto y se muestra. Si armara el objeto local con lo que escribió el usuario, ese campo faltaría o estaría mal. Por eso StockFlow guarda `saved`, el objeto que devolvió el backend: por ejemplo, el `id` del producto nuevo sólo existe en la respuesta del 201.

Pregunta 31

> *Explica Promise, async/await, try/catch/finally y dónde debe vivir cada responsabilidad.*

Para entender el concepto

Una **Promise** es un "vale" por un resultado que llegará más tarde: puede cumplirse (con el valor) o rechazarse (con un error). `async` marca una función que siempre devuelve una Promise; `await` espera el resultado **dentro de esa función** sin bloquear el navegador: mientras tanto la página sigue respondiendo a clics y animaciones.

> `try` envuelve el código que puede fallar, `catch` decide qué hacer si falla y `finally` se ejecuta siempre, haya salido bien o mal: ideal para "apagar" un indicador de carga.

Qué responder

- `apiFetch` es `async` y devuelve `Promise<T>`. Los services devuelven esa Promise sin esperarla. Quien necesita el resultado hace `await`.
- `await` **suspende la función**, no el hilo del navegador: el event loop sigue procesando eventos y renders.
- **Reparto de responsabilidades:**
  - `apiClient`: convierte respuestas fallidas en errores (`throw`); no muestra nada.
  - Hooks y Pages/Forms: `try/catch/finally` para traducir el error a estado de UI (mensaje, toast) y resetear flags.
  - `finally`: `setSaving(false)`, `setDeleting(false)`, `setLoadingDetailId(null)`, `setLoading(false)`.
- **Sin catch vacíos**: cada `catch` muestra algo o decide conscientemente ignorar (sólo el `AbortError`). El único `catch` sin cuerpo es el de `JSON.parse` en `construirError`, y está comentado: si el error no es JSON se usa el texto.

Muéstralo en el código

- `src/api/apiClient.ts:52` (`async function apiFetch<T>(): Promise<T>`) y `:36-43`.
- `src/features/productos/components/ProductoForm.tsx:107-124` (`try/catch/finally`).
- `src/shared/hooks/useAsyncList.ts:24-34`.

> **¿Qué ocurre con saving=true si haces return o throw y nunca lo restableces?**
> El botón queda para siempre en "Guardando..." y deshabilitado: el usuario no puede reintentar ni corregir, y la única salida es cerrar el modal o recargar. Por eso `setSaving(false)` está en `finally`, que se ejecuta aunque haya `return` o `throw` dentro del `try` o del `catch`. (El `return` por errores de validación ocurre antes de `setSaving(true)`, así que no lo afecta.)

Pregunta 32

> *Haz un diagnóstico ordenado cuando Swagger funciona pero React no.*

> **Para entender el concepto**
> Si Swagger funciona, el backend y la base están bien. El problema está en la **diferencia** entre lo que envía Swagger y lo que envía React. Diagnosticar es comparar las dos peticiones con evidencia (Network), no adivinar.

Qué responder

Abro DevTools → Network, reproduzco el fallo y comparo la petición real de React con la que funcionó en Swagger, en este orden:

1. **¿Salió la petición?** Si no aparece en Network, el problema está antes: validación del formulario, un `return` temprano o un error de JavaScript (Console).
2. **URL y base path**: ¿`VITE_API_URL` termina en `/api`? ¿El endpoint es `/productos` y no `/producto`? ¿Puerto 8080?
3. **Método**: ¿POST/PUT/DELETE correcto?

4. **CORS**: ¿la Console dice "blocked by CORS policy"? ¿El preflight `OPTIONS` falló? ¿Vite corre en 5173?
5. **Headers**: ¿`Content-Type: application/json` cuando hay body?
6. **Payload**: comparo el JSON de la pestaña Payload con el de Swagger: nombres de campos (`categoriaId`), tipos (número, no `"3"`), `null` vs `""`.
7. **Status y body de la respuesta**: 400 con `fieldErrors`, 404, 409, 422 → el mensaje dice qué falta.
8. **Forma de la respuesta**: si es 200 pero la pantalla falla, comparo el JSON con el modelo TypeScript (¿un campo renombrado?, ¿un objeto paginado en vez de array?).

Muéstralo en el código

- `.env.development` (base `http://localhost:8080/api`) y `src/config/env.ts`.
- `src/api/apiClient.ts:53-59` (headers y URL final).
- Swagger: `http://localhost:8080/swagger-ui.html`.

Si React llama /api/clientes pero Swagger muestra /clientes, ¿qué evidencia concreta verías en Network?

La petición de React aparecería con la URL `http://localhost:8080/api/clientes` y status **404** (el backend no tiene ese mapping), con el body de error de Spring indicando el `path` que no encontró. En Swagger, la petición exitosa iría a `/clientes`. La diferencia se ve comparando la columna "Name"/"Request URL" de ambas. La corrección va en un solo lugar: `VITE_API_URL` o la ruta del service. (En StockFlow el backend sí usa el prefijo `/api`, y Swagger lo muestra así.)

Bloque D — Relación 1:N, asincronía, hooks y UI avanzada

*Objetivo del bloque: comprobar que manejas relaciones, Effects, cancelación, custom hooks y componentes reutilizables sin duplicar responsabilidades. En StockFlow la relación es **Categoría (1) → Producto (N)**.*

Pregunta 33 · CRÍTICA

> *Representa Cliente 1:N Vehículo (Categoría 1:N Producto) desde el frontend y explica el papel de clienteId (categoriaId).*

> **Para entender el concepto**
>
> "1:N" significa que **una** categoría tiene **muchos** productos y cada producto pertenece a **una sola** categoría. En la base de datos esto se guarda en el lado "muchos": cada fila de `producto` tiene una columna `categoria_id` (la **clave foránea**, FK) que apunta a una fila de `categoria`.
>
> En el frontend no hace falta copiar la categoría completa dentro de cada producto: basta con el **identificador** (`categoriaId`), igual que una factura dice "cliente Nº 25" sin copiar toda la ficha del cliente.

Qué responder

- **Relación de dominio/backend**: en PostgreSQL, `producto.categoria_id` con la constraint `fk_producto_categoria`; en JPA, `@ManyToOne` en `ProductoJpaEntity` (objeto `CategoriaJpaEntity`, LAZY).
- **Contrato de UI**: el modelo `Producto` transporta sólo `categoriaId: number`; no contiene el objeto categoría. Las categorías se cargan aparte (`useCategorias`) y se cruzan por id cuando hace falta mostrar el nombre.
- **Al crear o editar**, el usuario elige la categoría por su nombre en un `<select>`, pero el payload envía `categoriaId`.
- **Quién valida la referencia**: el frontend sólo ayuda (la categoría debe existir en la lista y estar activa). El **backend** verifica que exista (404) y que esté activa (422), y PostgreSQL garantiza con la FK que nunca quede un producto apuntando a una categoría inexistente.
- Desde la categoría se navega a sus productos con `/productos?categoria=ID`, y la tabla de categorías muestra cuántos productos tiene cada una.

Muéstralo en el código

- `src/features/productos/models/Producto.ts:15-21` (comentario: "La relación con Categoria (1:N) viaja como categoriaId").
- `features/productos/components/ProductoForm.tsx:66-69` (comentario) y `:56` (`categoriaId: Number(data.categoriaId)`).
- `features/categorias/components/CategoriaTable.tsx:50` (enlace a los productos de la categoría).
- Base: `V1__creacion_completa_stockflow.sql:103` (`fk_producto_categoria`).

¿Por qué enviar clienteId (categoriaId) no significa que React "tenga una foreign key" igual que PostgreSQL?

Porque en React `categoriaId` es sólo un **número** dentro de un objeto: nada impide que valga 9999 o que la categoría se borre después. Una FK es una **restricción** que el motor de base de datos hace cumplir en cada INSERT, UPDATE y DELETE, incluso con peticiones simultáneas. React representa la relación; PostgreSQL la garantiza.

Pregunta 34

Explica el <select> controlado de propietario (categoría) y la conversión string → number.

Para entender el concepto

Un `<select>` controlado funciona igual que un input controlado: su `value` viene del estado y `onChange` lo actualiza. Las opciones (`<option>`) se generan a partir de una lista cargada del backend. El valor de un `<option>` siempre es **texto**, así que el id viaja como string hasta que se convierte en el payload.

Qué responder

- **Cargar antes de construir**: `ProductosPage` carga las categorías con `useCategorias()` y se las pasa al formulario por props (`categorias={categorias.data}`). Mientras cargan, la página muestra el esqueleto.
- **Opciones**: `opciones` = categorías activas + la categoría actual del producto aunque esté inactiva (para no perderla al editar). Cada opción usa `key={c.id}` y `value={String(c.id)}`. La primera opción es `value=""` ("Selecciona una categoría").
- **Controlado**: `<Select name="categoriaId" value={formData.categoriaId} onChange={handleChange}>`; el estado guarda el id como texto.
- **Validar y convertir**: `validarProducto` exige un valor no vacío, que exista en la lista y que esté activa (salvo que sea la categoría que el producto ya tenía). Recién después `aPayload` hace `Number(data.categoriaId)`.
- Si no hay categorías activas, el campo muestra la ayuda "No hay categorías activas: crea una primero".

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:80` (`opciones`), `:136-153` (Field + Select) y `:56` (conversión).
- `features/productos/utils/productoValidation.ts:22-28` (validación de la categoría).
- `features/productos/pages/ProductosPage.tsx:43` (`useCategorias()`) y `:305` (props al formulario).

¿Qué harías si el cliente (la categoría) seleccionado fue eliminado o desactivado justo antes del POST?

El frontend no puede detectarlo: su lista es de antes. El backend responde **404** si la categoría ya no existe o **422** si quedó inactiva. `ProductoForm` captura el `ApiError`, muestra el mensaje en el Alert del modal, pinta el error del campo si vino en `fieldErrors` y deja el formulario abierto con los datos, para que el usuario elija otra categoría. Lo ideal además es recargar la lista de categorías para que la opción vieja desaparezca.

Pregunta 35 · CRÍTICA

> *Cambiar propietario de un Vehículo (cambiar la categoría de un Producto): recorre el PUT completo y explica qué debe ocurrir en UI y backend.*

Para entender el concepto

Cambiar la relación de un hijo es simplemente **cambiar el valor de su clave foránea**: el producto pasa a apuntar a otra categoría. En el frontend eso es elegir otra opción en el select; en la base, un `UPDATE producto SET categoria_id = ...`. Lo delicado es la regla de negocio: la categoría nueva tiene que existir y estar activa.

Qué responder

1. **Precargar**: "Editar" → `GET /productos/{id}` → `aFormulario` pone `categoriaId: String(producto.categoriaId)`; el select muestra la categoría actual.
2. **Nueva selección**: el usuario elige otra categoría; `handleChange` guarda el nuevo id (texto).
3. **Validar**: `validarProducto(formData, categorias, producto.categoriaId)`. La regla es: un producto que **cambia** de categoría sólo puede ir a una activa; si conserva la suya, aunque esté inactiva, se permite.
4. **PUT**: `productoService.actualizar(id, { ...aPayload(formData), activo })` → `PUT /api/productos/{id}` con el nuevo `categoriaId` numérico.
5. **Backend**: busca el producto (404 si no existe), busca la categoría nueva (404 si no existe), verifica que esté activa (**422** si no), verifica que el código siga siendo único (409) y hace el UPDATE de `categoria_id`. Responde 200 con el producto actualizado.
6. **UI**: `handleSaved` reemplaza la fila con la respuesta (`prev.map`). La columna "Categoría" se recalcula sola, porque el nombre sale de `categoriasPorId.get(p.categoriaId)`. Si la tabla estaba filtrada por la categoría vieja, el producto deja de verse en ese filtro.
7. **Errores**: 404/422/409 se muestran en el Alert del modal y el formulario queda abierto.

Muéstralo en el código

- `src/features/productos/components/ProductoForm.tsx:38-41` (precarga), `:100` (validación con `producto?.categoriaId`) y `:110` (PUT).
- `features/productos/utils/productoValidation.ts:9-14` (comentario de la regla) y `:26-28`.
- `features/productos/components/ProductoTable.tsx:56` (nombre resuelto por id).
- Prueba: `features/productos/pages/ProductosPage.test.tsx:56-70` ("muestra la regla de negocio cuando el backend responde 422 al guardar").

Demuéstralo

Edita un producto y cámbiale la categoría. En Network → PUT → Payload verás `categoriaId` nuevo (número); en Response el producto con ese `categoriaId`. En DataGrip: `SELECT producto_id, codigo, categoria_id FROM stockflow.producto WHERE producto_id = X;`.

Si el PUT actualiza la placa (el código) pero no el propietario (la categoría), ¿qué comparaciones harías entre payload, DTO y respuesta?

1) **Payload** en Network: ¿viaja el `categoriaId` nuevo y como número? Si viaja el viejo, el problema está en el estado del formulario (el select no actualizó `formData`, por ejemplo un `name` distinto). 2) **DTO del backend** (`ActualizarProductoRequest`): ¿el campo se llama exactamente `categoriaId`? Si se llamara distinto, Jackson lo ignoraría sin error. 3) **Servicio/mapper**: ¿el caso de uso aplica el cambio de categoría? 4) **Respuesta**: ¿trae el `categoriaId` nuevo? Si la respuesta es correcta pero la tabla no cambia, el problema está en `handleSaved` o en cómo la tabla resuelve el nombre.

Pregunta 36

¿Cómo muestras el nombre del propietario (de la categoría) sin duplicarlo de forma peligrosa dentro de Vehículo (Producto)?

Para entender el concepto

Si cada producto guardara una copia del nombre de su categoría, al renombrar la categoría habría que actualizar todas las copias; si se olvida una, la pantalla muestra datos viejos. Es mejor guardar el nombre en **un solo lugar** (la lista de categorías) y "buscarlo" por id cuando se necesita, como consultar una agenda en lugar de anotar el teléfono en cada carta.

Un `Map` es una estructura de JavaScript que guarda pares clave → valor y permite buscar por clave de forma inmediata, sin recorrer toda la lista.

Qué responder

- `ProductosPage` construye `categoriasPorId = new Map(categorias.data.map(c => [c.id, c]))` con `useMemo` (se recalcula sólo cuando cambian las categorías).
- `ProductoTable` lo recibe por props y en cada fila hace `categoriasPorId.get(p.categoriaId)` para mostrar nombre y código. La búsqueda por texto también usa ese Map, así se puede buscar productos por el nombre de su categoría.
- Con `find` funcionaría igual, pero recorrería toda la lista por cada fila; el `Map` busca en tiempo constante.
- **Cuándo sería mejor un DTO con resumen**: si el backend devolviera productos de muchas categorías que no tengo cargadas, o con paginación en el servidor, convendría que `ProductoResponse` trajera `categoriaNombre` (un resumen de sólo lectura calculado por el backend con un JOIN). Para el tamaño actual, cruzar dos listas ya cargadas es suficiente.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:68-72` (Map con `useMemo`) y `:81-83` (búsqueda por nombre de categoría).
- `features/productos/components/ProductoTable.tsx:17` (prop) y `:56` (`categoriasPorId.get`).

Si cambia el nombre del Cliente (de la Categoría), ¿qué enfoque evita tener que editar todos los Vehículos (Productos) en memoria?

Resolver el nombre por id en el momento de renderizar. Al editar la categoría, sólo se actualiza **un** objeto en la lista de categorías; el `Map` se recalcula (por `useMemo` con dependencia `categorias.data`) y todas las filas muestran el nombre nuevo en el siguiente render. Ningún producto guarda el nombre, así que no hay copias que actualizar.

Pregunta 37

> *Filtrar Vehículos por Cliente (Productos por Categoría): compara filtrado frontend y endpoint filtrado en backend.*

Para entender el concepto

Filtrar en el frontend significa descargar todos los datos una vez y elegir en el navegador cuáles mostrar: es instantáneo, pero sólo sirve si los datos son pocos. **Filtrar en el backend** significa pedir sólo lo necesario (`?categoria=3`, `?pagina=2`): la base hace el trabajo con índices y viaja menos información, pero cada cambio de filtro es una petición nueva.

Qué responder

- En **Productos**, StockFlow filtra en el frontend: el catálogo es chico y ya está cargado completo, así que cambiar de categoría es inmediato y no genera peticiones. Además el filtro se combina con la búsqueda por texto y el estado.
- El filtro es **estado**, no una lista guardada: vive en la URL (`?categoria=ID`, leído con `useSearchParams`) y la lista filtrada se deriva. Ventaja extra: el enlace "Ver productos" desde Categorías abre la pantalla ya filtrada y se puede compartir.
- El backend **también** ofrece el endpoint filtrado `GET /api/productos/categoria/{categoriaId}`; sería la opción correcta si los productos fueran miles.
- Contraejemplo dentro del mismo proyecto: el **kardex** se pagina en el servidor (`inventarioService.kardex(productoId, pagina, tamano)`), porque los movimientos crecen sin límite. Ahí la respuesta trae el total real del servidor (`PaginaResponse`), no el largo de un array local.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.tsx:59` (filtro desde la URL), `:85` (comparación con `Number`) y `:93-96` (`setSearchParams`).
- Servidor: `features/inventario/services/inventarioService.ts:19` (kardex paginado) y `features/inventario/pages/MovimientosPage.tsx:42`.
- Backend: `ProductoController` → `@GetMapping("/categoria/{categoriaId}")`.
- Prueba: `ProductosPage.test.tsx:42` ("aplica el filtro de categoría que llega en la URL").

Con 100.000 vehículos (productos), ¿por qué descargar todo para luego filter() deja de ser razonable?

Porque se transferirían megas de JSON en cada carga, el navegador tardaría en parsearlos y guardarlos en memoria, cada tecla del buscador recorrería 100.000 elementos y la pantalla se trabaría (sobre todo en móviles). Además se descargarían datos que el usuario nunca verá. Lo correcto es filtrar y paginar en el backend con parámetros de consulta (la base usa índices) y mostrar el total real que informa el servidor.

Pregunta 38 · CRÍTICA

> *Explica useEffect con el modelo mental correcto: sincronización con sistemas externos.*

Para entender el concepto

Renderizar debe ser un cálculo puro: con las mismas props y estado, el mismo resultado. Pero a veces el componente tiene que **mantenerse en sintonía con algo fuera de React**: un servidor,

un temporizador, el foco del navegador, un `<dialog>` nativo. Para eso existe `useEffect`.

Tiene tres partes: el **setup** (conectar/iniciar), las **dependencias** (cuándo hay que volver a sincronizar) y el **cleanup** (desconectar/cancelar lo anterior). No es "una función que corre una vez": corre después de cada render en que cambió alguna dependencia.

Qué responder

- En `useAsyncList`, el Effect sincroniza la UI con la API:
  - **Setup**: crea un `AbortController`, pone `loading` y llama al loader.
  - **Dependencias** `[loader, reloadToken]`: se vuelve a sincronizar si cambia la función de carga o si alguien pide `reload()`.
  - **Cleanup** `() => controller.abort()`: si el componente se desmonta o el Effect se repite, cancela la petición anterior.
- Otros Effects reales: en `Modal` sincroniza la prop `open` con el `<dialog>` nativo (`showModal`/`close`); en `ProductoForm` pone el foco en el primer campo y su cleanup cancela el `requestAnimationFrame`; en `MainLayout`, al cambiar `pathname` mueve el foco al contenido.
- **No uso Effects** para calcular datos derivados (filtros, totales): eso se calcula en el render. Tampoco para responder a un clic: eso va en el handler del evento.

Muéstralo en el código

- `src/shared/hooks/useAsyncList.ts:21-38`.
- `src/components/ui/Modal.tsx:32-37`.
- `src/features/productos/components/ProductoForm.tsx:82-85`.
- `src/layouts/MainLayout.tsx:62-64`.

Si una dependencia usada dentro del Effect cambia y no está declarada, ¿qué tipo de bug puede aparecer?

Un bug de **datos desactualizados** (stale closure): el Effect sigue usando el valor viejo que capturó y no vuelve a sincronizar. Ejemplo: si `useKardex` no declarara `pagina`, al pasar a la página 2 seguiría mostrando la página 1. Por eso todas las variables que el Effect lee deben estar en su lista de dependencias.

Pregunta 39 · CRÍTICA

Explica cleanup, AbortController y cómo evitas respuestas obsoletas/race conditions.

Para entender el concepto

Las respuestas de red no llegan necesariamente en el orden en que se pidieron. Si pides la página 1 y enseguida la página 2, y la 1 tarda más, la respuesta vieja puede llegar **después** y pisar a la nueva: es una **condición de carrera** (race condition).

`AbortController` es una API del navegador para **cancelar** un `fetch`: creas un controlador, pasas su `signal` a `fetch` y, cuando llamas a `controller.abort()`, esa petición se cancela y su Promise se rechaza con un `AbortError`.

Qué responder

- **Crear y pasar la señal**: `useAsyncList` crea `new AbortController()` en cada ejecución del Effect y pasa `controller.signal` al loader → service → `apiFetch` → `fetch`.
- **Abortar en el cleanup**: `return () => controller.abort()`. Se ejecuta al desmontar la pantalla o antes de repetir el Effect (por ejemplo, con `reload()`).
- **AbortError no es un error funcional**: `isAbortError(err)` lo reconoce (`DOMException` con `name === 'AbortError'`) y no se muestra mensaje. Además `setLoading(false)` sólo se ejecuta si la señal no fue abortada, para no apagar el indicador de una petición más nueva.
- **Descartar lo que llega tarde**: abortar no alcanza siempre. Si la respuesta ya venía en camino cuando se abortó (el servidor ya había respondido), la Promise puede resolverse igual. Por eso, antes de guardar, los dos hooks preguntan `if (!controller.signal.aborted)`: si la petición fue abortada, el resultado se descarta y no pisa al actual. Es la misma idea que la bandera `ignore` de la documentación de React.
- **El mejor ejemplo es `useKardex`**: cada combinación producto/página/tamaño genera una `key`; el Effect aborta la petición anterior, descarta su respuesta si llega igual y guarda cada resultado junto con la key que lo produjo. "Cargando" se **deriva** (la key actual todavía no tiene resultado) y, mientras llega la página nueva, se sigue mostrando la anterior atenuada.
- **Está probado**: `useKardex.test.ts` resuelve primero la página 1 y *después* la página 0, y comprueba que la pantalla muestra la 1. Esa prueba encontró un error real: antes de agregar el `if (controller.signal.aborted)`, la página 0 pisaba a la 1 y la pantalla quedaba en "cargando".

Muéstralo en el código

- `src/shared/hooks/useAsyncList.ts:22` (controller), `:27` (signal al loader), `:29` (descarta si se abortó), `:31` (ignora AbortError), `:33` (loading sólo si no abortó), `:37` (cleanup).
- `src/shared/utils/isAbortError.ts:2-4`.
- `src/features/inventario/hooks/useKardex.ts:9-51` (comentario y lógica con `key`) y `:31` (descarta la respuesta abortada).
- Pruebas: `src/features/inventario/hooks/useKardex.test.ts:27` y `src/shared/hooks/useAsyncList.test.ts:62` (respuesta vieja que llega tarde).
- `features/productos/services/productoService.ts:7-9` (el service recibe y pasa `signal`).

Demuéstralo

En Network activa "Slow 3G", entra a Productos y cambia rápido a Categorías: la petición de productos aparece como *(canceled)* y no hay mensajes de error en pantalla. En Movimientos, cambia de página varias veces rápido: las anteriores se cancelan y sólo se muestra la última. Para demostrar que la prueba protege la regla: borra la línea 31 de `useKardex.ts` y corre `npx vitest run src/features/inventario/hooks`: falla "descarta la respuesta de la página anterior aunque llegue después". Deshaz el cambio.

Si buscas A y enseguida B, pero la respuesta de A llega después, ¿cómo evitas mostrar A como si fuera el resultado actual?

Con dos defensas que StockFlow combina: 1) **cancelar** la petición de A en el cleanup cuando el parámetro cambia a B (`controller.abort()`); normalmente su Promise se rechaza con `AbortError` y se ignora. 2) **Descartar** lo que llegue igual: si A ya venía en camino y se resuelve, el `.then` ve `controller.signal.aborted === true` y no guarda nada (`useKardex.ts:31`). Además `useKardex` etiqueta cada resultado con su key para derivar "cargando". `useKardex.test.ts:27` lo demuestra con A llegando después de B.

Pregunta 40

> *¿Por qué un Effect puede ejecutarse dos veces en desarrollo con StrictMode y qué debes hacer?*

Para entender el concepto

`StrictMode` es una herramienta de React **sólo para desarrollo**. Para ayudarte a encontrar Effects mal escritos, monta el componente, simula desmontarlo (ejecuta el cleanup) y lo vuelve a montar: setup → cleanup → setup. Si tu Effect está bien escrito (el cleanup deshace lo que hizo el setup), no notarás diferencia. Si no, el problema aparece enseguida en vez de en producción.

Qué responder

- StockFlow envuelve la app en `<StrictMode>`. En desarrollo, al entrar a Productos se ven **dos** GET en Network: el primero aparece como *(canceled)* porque el cleanup lo abortó, y el segundo completa. En producción (`npm run build`) hay uno solo.
- Lo correcto es **corregir el Effect** (cleanup y operaciones idempotentes), no desactivar StrictMode. Nuestros Effects ya lo cumplen: `useAsyncList` aborta en el cleanup, `ProductoForm` cancela el `requestAnimationFrame` y `Modal` comprueba `dialog.open` antes de abrir o cerrar.
- StrictMode también renderiza dos veces los componentes en desarrollo para detectar renders impuros.

Muéstralo en el código

- `src/main.tsx:13-16` (comentario y `<StrictMode>`).
- `src/shared/hooks/useAsyncList.ts:37` y `src/components/ui/Modal.tsx:35-36`.

Si tu Effect crea una suscripción y no la limpia, ¿qué síntoma puede revelar StrictMode?

Quedan **dos suscripciones activas** a la vez: el mismo evento se procesa dos veces (dos peticiones, dos toasts, contadores que suben de a dos), hay fugas de memoria y, si la pantalla ya se cerró, aparecen actualizaciones de estado en un componente desmontado. StrictMode lo hace visible al instante porque monta, "desmonta" y vuelve a montar.

Pregunta 41

> *¿Qué problema resuelven useAsyncList<T>, useClientes y useVehiculos (en StockFlow: useAsyncList<T>, useCategorias y useProductos)?*

Para entender el concepto

Un **custom hook** es una función que empieza con `use` y agrupa lógica de React (estado, efectos) para reutilizarla. Ojo: reutiliza la **lógica**, no el **estado**. Cada componente que llama al hook obtiene su propia copia independiente del estado, como dos personas que usan la misma receta: cada una cocina su propio plato.

La `<T>` es un **genérico**: un "tipo comodín" que se reemplaza por el tipo real al usarlo (`useAsyncList<Producto>`).

Qué responder

- **Problema**: todas las pantallas repetían la misma mecánica: estado de datos, de carga y de error; un Effect que llama a la API; cancelación con AbortController; ignorar el AbortError; poder recargar.

- **`useAsyncList<T>`** (genérico, en `shared/`): gestiona la mecánica para cualquier colección. Devuelve `data`, `setData` (para actualizar tras POST/PUT/DELETE), `loading`, `error`, `setError` y `reload` (refetch cambiando `reloadToken`).
- **`useCategorias`** / **`useProductos`** (de dominio, en cada feature): expresan "dame las categorías / los productos" usando el service del módulo. La Page no sabe de AbortController ni de URLs.
- **`useResumenInventario`** combina tres de ellos (categorías, productos, existencias) para Inicio y Dashboard, con un `loading` y un `error` comunes.
- **Requisito importante**: el loader debe ser una referencia estable (definida fuera del componente, como `productoService.listar`); si fuera una función nueva en cada render, el Effect se repetiría sin parar.

Muéstralo en el código

- `src/shared/hooks/useAsyncList.ts:7-40` (comentario y hook).
- `src/features/productos/hooks/useProductos.ts` y `features/categorias/hooks/useCategorias.ts`.
- `src/features/inventario/hooks/useResumenInventario.ts:7-24`.

Si dos componentes llaman al mismo custom hook, ¿comparten siempre el mismo useState? ¿Por qué?

No. Cada llamada a un hook crea su propio estado, ligado al componente que lo llama. `ProductosPage` y `DashboardPage` llaman cada una a `useProductos()` y obtienen listas independientes (cada una hace su propio GET). Para compartir estado entre componentes hay que subirlo a un ancestro común y pasarlo por props, usar un Context o una librería de server state (por ejemplo TanStack Query, que además cachea).

Pregunta 42

Defiende tus componentes UI compartidos, useMemo, design tokens y decisiones de accesibilidad.

Para entender el concepto

- **Componentes UI compartidos**: piezas visuales que se escriben una vez y se configuran con props (variantes, tamaños). Dan coherencia y evitan copiar y pegar.
- **`useMemo`**: guarda el resultado de un cálculo y sólo lo repite si cambian sus dependencias. Es una **optimización**: el código debe funcionar igual sin él.
- **Design tokens**: variables CSS con nombres de significado (`--surface`, `--text-muted`) en vez de colores sueltos. Cambiar el tema es cambiar los tokens.
- **Accesibilidad**: que la app se pueda usar con teclado, con lector de pantalla y con buen contraste.

Qué responder

- **UI compartida** en `components/ui/`: `Button` (variantes `primary/secondary/danger`, tamaños, `loading` con `loadingText` y `aria-busy`), `Field` (label + control + ayuda + error), `SearchInput`, `SelectFilter`, `Pagination`, `Modal`, `ConfirmDialog`, `DataTable` (genérica con `rowKey`), `StatusBadge`, `EmptyState`, `Alert`, toasts. Ninguna importa nada de `features/`.
- **useMemo**: lo uso en `ProductosPage` para el `Map` de categorías y el stock por producto, que se recorren en cada fila. La lista filtrada no lo usa porque es barata de calcular.

- **Tokens**: `styles/tokens.css` define los nombres semánticos, con valores para tema claro y oscuro; los componentes nunca usan colores directos.
- **Accesibilidad**:
  - `Field` enlaza label e input con `useId` + `htmlFor`, y el error y la ayuda con `aria-describedby`; marca `aria-invalid`. Al fallar la validación, el foco va al primer campo inválido.
  - `Modal` usa `<dialog>` + `showModal()` (foco atrapado, Escape, foco devuelto) y `aria-labelledby`.
  - `Pagination` es un `<nav aria-label="Paginación">` con `aria-current="page"` y botones deshabilitados en los extremos.
  - Botones reales (`<button>`) en lugar de `div` clicables; íconos con `aria-hidden` y botones de sólo ícono con `aria-label`.
  - Los esqueletos de carga son `role="status"` con texto para lectores de pantalla; al cambiar de ruta el foco va al contenido principal.

Muéstralo en el código

- `src/components/ui/Button.tsx:25-42`, `Field.tsx:21-39`, `Pagination.tsx:25-56`, `Modal.tsx:21-44`, `LoadingSkeleton.tsx:17-18`.
- `src/features/productos/pages/ProductosPage.tsx:69-76` (`useMemo`).
- `src/styles/tokens.css` (comentario de la línea 2).
- `src/layouts/MainLayout.tsx:62-64` (foco al cambiar de ruta).

¿Cómo demostrarías que un modal funciona para un usuario que navega sólo con teclado?

Sin tocar el mouse: Tab hasta "Nuevo producto" y Enter → el modal abre y el foco cae en el primer campo; Tab recorre sólo los controles del modal (no se escapa al fondo); Escape lo cierra; al cerrarse, el foco vuelve al botón "Nuevo producto". También se puede comprobar con el lector de pantalla o con el panel de Accesibilidad de DevTools que el diálogo se anuncia con su título. Hay pruebas automáticas: `Modal.test.tsx` busca el diálogo por su rol y nombre (`getByRole('dialog', { name })`).

Bloque E — Testing, build, producción y defensa integral

Objetivo del bloque: verificar calidad, despliegue y dominio de punta a punta, más allá de que el CRUD "funcione".

Pregunta 43 · CRÍTICA

> *Diseña la estrategia de testing de ParkFlow360 (StockFlow) por niveles.*

Para entender el concepto

Las pruebas automáticas son programas que verifican que tu programa hace lo que debe, y que lo siga haciendo cuando cambias algo (detectan **regresiones**). Se organizan por **niveles**: abajo, muchas pruebas pequeñas y rápidas de funciones sueltas; en el medio, pruebas de componentes y pantallas como las usaría un usuario; arriba, pocas pruebas lentas que conectan con el sistema real. Cada nivel encuentra un tipo distinto de error.

Qué responder

Nivel	En StockFlow (30 archivos, 107 pruebas)	Qué riesgo detecta
1. Lógica pura	8 archivos: `productoValidation`, `categoriaValidation`, `ubicacionValidation`, `loginValidation`, `paginar`, `csv`, `indicadores`, `stockPorProducto`	Reglas mal programadas: una validación que deja pasar un stock negativo, una paginación que se sale de rango, un cálculo de indicador incorrecto
2. Componentes UI	8 archivos: `Modal`, `Pagination`, `SearchInput`, `StatusBadge`, `Tabs`, `Dropdown`, `ToastProvider`, `HeaderSearch`	Accesibilidad e interacción: un botón que no se deshabilita, un modal sin nombre accesible, un callback que no se emite
3. Páginas con interacción (services simulados)	10 archivos: `CategoriasPage`, `ProductosPage`, `InventarioPages`, `AlmacenesPage`, `ReportesPage`, `LoginPage`, etc.	Flujos de usuario y manejo de errores: que un 409 no borre la fila, que el 422 se muestre, que el filtro de la URL se aplique
4. Hooks y services	3 archivos: `useAsyncList.test.ts`, `useKardex.test.ts` (con `renderHook`) e `inventarioService.test.ts` (con `fetch` simulado)	Asincronía y contrato de rutas: una respuesta vieja que pisa a la nueva, un `AbortError` mostrado como error, un cleanup que no aborta, una URL o query mal armada (`?pagina=&tamano=`)
5. Cliente HTTP (`fetch` simulado)	`apiClient.test.ts`	Contrato de errores: que un 204 no intente leer JSON, que un error del backend se convierta en `ApiError` con status y `fieldErrors`

6. Integración real con el backend	**No automatizada todavía**: se cubre con la colección Postman, las pruebas Testcontainers del backend y la verificación manual con Network	Contrato real: nombres de campos, CORS, rutas, comportamiento del backend

- **No todo se prueba mockeando ni todo requiere backend real**: los mocks dan rapidez y determinismo en los niveles 2-5; la integración real es necesaria para detectar que el contrato cambió.

Muéstralo en el código

- `src/features/productos/utils/productoValidation.test.ts`, `src/components/ui/Pagination.test.tsx`, `src/features/categorias/pages/CategoriasPage.test.tsx`, `src/shared/hooks/useAsyncList.test.ts`, `src/features/inventario/hooks/useKardex.test.ts`, `src/api/apiClient.test.ts`.
- Configuración: `vite.config.ts:23-29` (jsdom + setup) y `src/test/setup.ts`.

Demuéstralo

`npm run test:run` → "Test Files 30 passed · Tests 107 passed". Para mostrar que una prueba protege una regla: en `productoValidation.ts` cambia `/^\d+$/` por `/^-?\d+$/` (aceptar negativos) y vuelve a correr: falla "RN-07: el stock mínimo debe ser un entero no negativo". Deshaz el cambio.

Dame un bug de UI que detectaría RTL y un bug de contrato HTTP que un test puramente presentacional no detectaría.

Bug de UI: que al recibir un 409 en el DELETE la tabla quite la fila igual. `CategoriasPage.test.tsx:52` lo detecta porque comprueba que la fila sigue visible y que aparece el mensaje. **Bug de contrato**: que el backend renombre `categoriaId` a `idCategoria` en `ProductoResponse`. Las pruebas de páginas siguen pasando porque el mock devuelve el formato viejo, pero en la app real la columna Categoría queda vacía. Eso sólo lo detecta una prueba contra el backend real (o una prueba de contrato).

Pregunta 44

Explica Vitest, React Testing Library y userEvent desde la perspectiva del usuario.

Para entender el concepto

- **Vitest**: el "motor" de pruebas. Encuentra los archivos `*.test.ts(x)`, los ejecuta y compara resultados con `expect(...)`. Usa la misma configuración que Vite.
- **jsdom**: un navegador simulado (sin pantalla) para que existan `document` y `window` en las pruebas.
- **React Testing Library (RTL)**: monta componentes y los consulta **como lo haría una persona**: por su rol ("botón Guardar"), su etiqueta o su texto, no por clases CSS ni estado interno.
- **userEvent**: simula interacciones reales (clic, escribir, tabular) con todos los eventos intermedios que dispara un navegador.

Qué responder

- Vitest ejecuta y hace las aserciones; `@testing-library/jest-dom` agrega matchers legibles como `toBeInTheDocument()` o `toBeDisabled()`.
- RTL renderiza la página con los mismos providers que la app (`renderWithProviders`: router en memoria, sesión demo y toasts) y busca elementos con **queries accesibles**: `findByRole('button', { name: 'Acciones de …' })`, `getByRole('menuitem', { name: 'Editar' })`, `findByRole('dialog', { name: 'Editar …' })`.
- `userEvent.click` simula al usuario: abre el menú de la fila, elige Editar y pulsa "Guardar cambios".
- La aserción final comprueba **lo que el usuario ve**: el título "La operación no cumple una regla del inventario" y el mensaje del backend dentro del modal.
- Preferir rol/label/texto también verifica la accesibilidad: si un botón no tuviera nombre accesible, la prueba no lo encontraría.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.test.tsx:56-70` (flujo completo con `userEvent`).
- `src/test/renderWithProviders.tsx:8-16` y `src/test/setup.ts` (matchers, limpieza y simulación de `<dialog>`, que jsdom no implementa).

¿Por qué probar que "setState fue llamado" suele ser peor que comprobar lo que el usuario ve después del clic?

Porque prueba un **detalle de implementación**: si mañana cambio `useState` por `useReducer` o muevo el estado a un hook, la prueba se rompe aunque la pantalla funcione igual (falso negativo). Y al revés: `setState` pudo llamarse con un valor equivocado o la pantalla pudo no mostrarlo, y la prueba pasaría igual (falso positivo). Comprobar lo visible prueba el comportamiento que importa y sobrevive a los refactors.

Pregunta 45

> *¿Cuándo mockearías la API y cuándo necesitas una integración más realista?*

Para entender el concepto

Un **mock** es un "doble de prueba": reemplaza una dependencia real (la API) por una versión falsa que responde lo que tú decides. Hace las pruebas rápidas, repetibles y capaces de simular casos difíciles (un 500, un 409). Pero un mock sólo sabe lo que tú le dijiste: si el backend real cambia, el mock no se entera.

Qué responder

- **Mockear** cuando pruebo el comportamiento del frontend ante respuestas conocidas: estados de carga, errores 409/422/500, que la fila se conserve. En StockFlow las pruebas de páginas usan `vi.mock('../services/productoService')` y `vi.mocked(...).mockResolvedValue / mockRejectedValue`. `apiClient.test.ts` simula `fetch` con `vi.stubGlobal` para probar la conversión de respuestas.
- **Integración real** cuando lo que quiero verificar está del otro lado: que los nombres de campos coincidan con los DTO, que CORS permita el origen, que las rutas existan, que el backend aplique las reglas. Hoy eso se cubre con la colección Postman, las pruebas de integración del backend (Testcontainers) y la verificación manual con Network.
- **MSW** (Mock Service Worker) sería un punto intermedio: intercepta las peticiones a nivel de red, así se prueba también `apiClient` y las URLs reales. StockFlow no lo usa; simula los services.

- **Límites**: el mock no detecta cambios de contrato; la integración real es más lenta, necesita backend y base levantados y es menos determinista.

Muéstralo en el código

- `src/features/productos/pages/ProductosPage.test.tsx:14-29` (`vi.mock` y respuestas por defecto) y `:48-50` (simular un 500).
- `src/api/apiClient.test.ts:5-10` (`vi.stubGlobal('fetch', ...)`).
- Fixtures compartidas: `features/productos/data/productos.mock.ts`, `categorias/data/categorias.mock.ts`.

Si todos los tests mockean una respuesta perfecta que ya no coincide con el backend, ¿qué falsa confianza aparece?

Todo queda en verde mientras la app real está rota: los tests prueban contra un backend imaginario. Es lo que pasaría en StockFlow si el backend cambiara un campo de `ProductoResponse`: los mocks de `productos.mock.ts` seguirían con el formato viejo. Por eso hace falta al menos una verificación contra el backend real (Postman, una prueba de contrato o una e2e) y mantener los mocks tipados con los mismos modelos (`Producto[]`), para que TypeScript avise si el modelo cambia.

Pregunta 46

Diferencia ErrorBoundary de manejo de errores de fetch.

Para entender el concepto

Un **ErrorBoundary** es una "red de seguridad" para errores que ocurren **mientras React dibuja** la pantalla (por ejemplo, leer una propiedad de `undefined` en el JSX). Si un componente explota al renderizar, el boundary muestra una pantalla alternativa en vez de dejar todo en blanco.

Pero no ve los errores que ocurren **fuera del render**: los de un `fetch` (son asíncronos) ni los de un handler de clic. Esos se manejan con `try/catch` y se convierten en **estado** (un mensaje en pantalla).

Qué responder

- `AppErrorBoundary` es un componente de clase (los boundaries todavía requieren `getDerivedStateFromError` y `componentDidCatch`). Envuelve toda la app en `main.tsx`; si un render falla, muestra "La interfaz tuvo un problema" con `role="alert"` y registra el error en consola.
- Los **errores HTTP** son esperables y se representan como estado controlado: `apiFetch` lanza `ApiError`, el hook o el formulario lo captura y la pantalla muestra un `Alert` o un toast con opción de reintentar. La app sigue funcionando.
- El boundary no es un sistema universal de excepciones: si lo usara para errores de red, un 409 haría desaparecer toda la pantalla en lugar de mostrar "código repetido" junto al campo.

Muéstralo en el código

- `src/app/AppErrorBoundary.tsx:6-31` (el comentario aclara: "NO captura errores de fetch ni de event handlers").
- `src/main.tsx:17` (dónde envuelve).
- Errores de fetch como estado: `src/shared/hooks/useAsyncList.ts:30-31` y `features/productos/components/ProductoForm.tsx:114-121`.

Si fetch responde 500 y tu service lo convierte en Error controlado, ¿por qué no deberías depender sólo de ErrorBoundary?

Porque el error ocurre en código asíncrono (dentro de un Effect o de un handler), fuera del render, y el boundary **no lo ve**: la promesa rechazada quedaría sin manejar y la pantalla seguiría "cargando" para siempre. Y aunque se forzara a llegar al boundary, se perdería toda la pantalla por un error recuperable. Lo correcto es `try/catch` → estado de error → mensaje con "Reintentar", como hace StockFlow.

Pregunta 47 · CRÍTICA

> *Explica npm run build, dist, npm run preview y variables de entorno por modo.*

Para entender el concepto

`npm run build` convierte el proyecto en **archivos estáticos** (HTML, JS, CSS) listos para subir a cualquier servidor web. Esos archivos van a la carpeta `dist/`. Lo importante: las variables `VITE_*` se **copian dentro del JavaScript en el momento del build**. El build es una "foto" de la configuración de ese momento; cambiar el `.env` después no cambia la foto.

Qué responder

- `npm run build` = `tsc -b && vite build`: verifica tipos, empaqueta, minifica y divide en trozos (cada página lazy es un `.js` aparte). Usa el **modo production**, que lee `.env.production`.
- `dist/` contiene `index.html` y `assets/` con nombres que llevan un hash (`apiClient-VTyo5whh.js`) para que el navegador no use una versión vieja en caché. Es un build estático: no necesita Node para funcionar, cualquier servidor de archivos lo sirve.
- `npm run preview` sirve `dist/` en local para **inspeccionar** el resultado antes de publicarlo. No es un servidor de producción. En StockFlow reenvía `/api` al backend local (`vite.config.ts:13-22`), así el build se prueba con datos reales.
- **Variables por modo**: `npm run dev` → `.env.development` (`http://localhost:8080/api`); `npm run build` → `.env.production`. Antes de compilar hay que verificar que `VITE_API_URL` de producción apunte al backend correcto.
- **En StockFlow**, `.env.production` tiene `VITE_API_URL=/api`: una URL **relativa**. El navegador la completa con el dominio desde donde se cargó la página, y el servidor que sirve `dist/` (`nginx.conf`) reenvía `/api/` al backend. Así frontend y API comparten origen y no hace falta inventar un dominio para la API. Si la API se publicara en otro dominio, habría que poner su URL completa y reconstruir.

Muéstralo en el código

- `package.json` → scripts `build` y `preview`.
- `.env.production` (`/api`) y `.env.development`; `src/config/env.ts`.
- `vite.config.ts:13-22` (proxy de `preview`) y `nginx.conf` (proxy de producción).
- Carpeta `dist/` (ignorada por Git en `.gitignore`).

Demuéstralo

`npm run build` → muestra `dist/assets`. Busca la URL embebida: `grep -o 'var e=`/api`' dist/assets/apiClient-*.js` (Vite reemplazó `import.meta.env.VITE_API_URL` por el texto `/api`). Luego, con el backend encendido, `npm run preview` y abre `http://localhost:4173/productos`: carga datos reales.

Si cambias .env.production después de haber generado dist pero no reconstruyes, ¿qué URL puede quedar embebida?

La **vieja**, la que estaba al momento del build. Vite reemplazó `import.meta.env.VITE_API_URL` por el texto literal dentro de `dist/assets/*.js`; el `.env` ya no se lee. Por ejemplo, un `dist/` generado antes del cambio a `/api` seguiría apuntando al dominio de ejemplo anterior (`api.stockflow.example.com`). Hay que ejecutar `npm run build` otra vez después de cada cambio de configuración.

Pregunta 48

BrowserRouter en producción: explica fallback SPA, rutas directas y CORS productivo.

Para entender el concepto

En una SPA sólo existe **un** archivo HTML real: `index.html`. Rutas como `/productos` las inventa React Router en el navegador. Si el usuario entra directo a `https://midominio.com/productos` (o recarga ahí), el navegador le pide al **servidor web** un archivo `/productos` que no existe → 404. El **fallback SPA** es una regla del servidor: "si no encuentras el archivo, devuelve `index.html`"; React arranca, lee la URL y muestra la pantalla correcta.

Qué responder

- Con `BrowserRouter` las URLs son "limpias" (`/productos`), así que el hosting debe tener fallback a `index.html` para cualquier ruta que no sea un archivo de `assets/`. En desarrollo Vite ya lo hace solo; en producción lo configura quien sirve `dist/`:

```
nginx:   location / { try_files $uri $uri/ /index.html; }
Netlify: archivo public/_redirects →  /*  /index.html  200
Vercel:  vercel.json → { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

- **StockFlow lo resuelve con `nginx.conf`**: `location / { try_files … /index.html; }` para el fallback, y `location /api/ { proxy_pass http://backend:8080; }` para la API.
- **CORS productivo**: con el proxy, el navegador ve **un solo origen** y no hay CORS. nginx reenvía el `Host` con el puerto (`$http_host`) porque Spring compara `Origin` con el `Host` para decidir si la petición es de otro origen: con `$host` (sin puerto) un POST del mismo origen daba 403 "Invalid CORS request". Si la API se publica en otro dominio, o nginx sirve por HTTPS (hacia el backend habla HTTP), el backend debe permitir el origen real del frontend en `CORS_ALLOWED_ORIGINS`; nunca `*` por comodidad.

- **Dos 404 distintos**: el 404 del hosting (no hay fallback: se pidió un documento HTML para `/productos`) y el 404 de la API (un endpoint o recurso inexistente en `/api/...`, que llega como JSON y lo muestra la app). En Network el primero es de tipo *document* y el segundo *fetch*.
- La pantalla de StockFlow para rutas inexistentes dentro de la app (`NotFoundPage`, `path="*"`) sólo funciona si el servidor primero entrega `index.html`.

Muéstralo en el código

- `src/main.tsx:18` (`BrowserRouter`) y `src/routes/AppRouter.tsx:51` (`path="*"`).
- Backend: `application.yml` → `stockflow.cors.allowed-origins: ${CORS_ALLOWED_ORIGINS:http://localhost:5173}`.
- `webstorm/stockflow-frontend/nginx.conf`: `location /api/` (proxy, `Host $http_host`) y `location /` (`try_files`).
- README del frontend, sección "Producción": prueba con Docker.

Demuéstralo

Con el backend encendido: `npm run build` y luego `docker run --rm -p 8081:80 --add-host backend:host-gateway -v "$PWD/dist:/usr/share/nginx/html:ro" -v "$PWD/nginx.conf:/etc/nginx/conf.d/default.conf:ro" nginx:1.27-alpine`. Abre `http://localhost:8081/productos` y presiona F5: la pantalla vuelve a cargar (es la "prueba obligatoria" de la guía G10). En Network, las llamadas van a `localhost:8081/api/...`.

Si / funciona pero al refrescar /vehiculos (/productos) el servidor devuelve 404, ¿qué configuración falta probablemente?

El **fallback SPA** del servidor web: la regla que devuelve `index.html` para rutas que no son archivos (`try_files ... /index.html` en nginx, `rewrites` en Vercel, `_redirects` en Netlify). `/` funciona porque ahí sí existe `index.html`; `/productos` no existe como archivo. No es un problema de React Router ni del backend.

Pregunta 49

Explica el Dashboard y por qué sus métricas son estado derivado, no una segunda fuente de verdad.

Para entender el concepto

Un dashboard muestra resúmenes: totales, porcentajes, rankings. Si esos números se guardaran aparte (un contador que se suma o resta a mano), habría dos verdades que tarde o temprano no coinciden. Si se **calculan** a partir de los datos actuales cada vez que se muestran, siempre son correctos.

Qué responder

- `DashboardPage` obtiene tres colecciones reales con `useResumenInventario` (categorías, productos y existencias de `/api/inventario/stock`) y **calcula** en cada render: productos activos, existencias bajo mínimo y su porcentaje, existencias por ubicación, valor referencial y productos por categoría.
- No hay contadores en `useState`: si cambia la fuente, las métricas cambian solas.
- "Actualizar" hace refetch de las tres colecciones; si alguna falla se muestra "Parte de la información no se pudo cargar".

- **Rendimiento y frescura**: con pocos datos calcular en el navegador es suficiente. Si crecieran, lo correcto sería un endpoint agregado en el backend (el propio comentario de `DashboardPage` lo anticipa) que haga el `GROUP BY` en PostgreSQL. Los datos se refrescan al entrar o con "Actualizar", no en tiempo real.

Muéstralo en el código

- `src/pages/DashboardPage.tsx:20-32` (comentario y cálculos).
- `src/features/inventario/hooks/useResumenInventario.ts:7-24`.
- Funciones de cálculo con pruebas: `features/inventario/utils/indicadores.ts` e `indicadores.test.ts`.

Si eliminas un Vehículo (Producto) y el contador no cambia, ¿qué dependencia o fuente de datos revisarías?

El Dashboard tiene su **propia** llamada a `useProductos` (los hooks no comparten estado): el borrado en la pantalla Productos no actualiza la lista del Dashboard hasta que se vuelve a montar o se pulsa "Actualizar". Revisaría: 1) que el Dashboard haga refetch al entrar (se monta de nuevo al navegar); 2) que la métrica se calcule de `productos` y no de un valor guardado; 3) si uso `useMemo`, que sus dependencias incluyan la colección; 4) en Network, que el GET devuelva la lista sin el producto borrado.

Pregunta 50 · CRÍTICA

Defiende el flujo fullstack completo sin saltos: desde una acción del usuario hasta PostgreSQL y de regreso a la UI.

Para entender el concepto

Esta pregunta junta todo: hay que contar una operación real como una historia continua, sin "y luego mágicamente...". Conviene elegir **Editar Producto cambiando su categoría**, porque toca formulario, relación 1:N, PUT, reglas de negocio y actualización de la tabla.

Qué responder

Frontend (ida)

1. **Componente**: en `ProductoTable` el usuario elige Editar → `onEdit(p)` → `ProductosPage.handleEdit` → `GET /api/productos/{id}` → abre el modal con `ProductoForm` precargado.
2. **Estado**: el usuario cambia la categoría en el select controlado → `handleChange` → `setFormData` → re-render.
3. **Validación**: "Guardar cambios" → `handleSubmit` → `preventDefault` → `validarProducto` (categoría existente y activa, código, stock, precio).
4. **Service**: `aPayload` + `activo` → `productoService.actualizar(id, data)`.
5. **apiClient/fetch**: `apiFetch` arma `http://localhost:8080/api/productos/{id}`, `Content-Type: application/json`, body JSON → `fetch` (preflight `OPTIONS` + `PUT`).

Backend (ya implementado en el parcial 1)

6. CORS permite el origen → `DispatcherServlet` → `ProductoController` (`@PutMapping`) → Jackson + `@Valid` → `ProductoService` (`@Transactional`): verifica producto (404), categoría (404), categoría activa (422), código único (409) → adaptador de persistencia → Hibernate → `UPDATE producto SET categoria_id = …` en PostgreSQL (la FK valida la referencia) → commit → 200 con `ProductoResponse`.

Frontend (vuelta)

7. `apiFetch` verifica `response.ok` → `response.json()` → `onSaved(producto, 'edit')`.
8. **Estado**: `productos.setData(prev => prev.map(...))`, cierra el modal, toast "Producto X actualizado".
9. **Render**: la Page recalcula filtros y página; la fila muestra la nueva categoría resuelta con `categoriasPorId`.

Dónde se rompería el flujo

Falla	Qué se ve	Capa que lo detecta
URL incorrecta	Network: 404 en `/api/...` (si el prefijo no es `/api`, además aparece como error CORS)	Backend (no hay mapping) → `ApiError` → Alert
CORS	Console "blocked by CORS policy"; Network "CORS error"	Navegador → `TypeError` → "No se pudo conectar..."
Payload inválido	400 con `fieldErrors` (si pasó la validación de UI)	Bean Validation → errores junto a los campos
Producto o categoría inexistente	404	Servicio → "El registro ya no existe" + `reload()`
Código repetido	409	Servicio/UNIQUE → mensaje en el modal
Categoría inactiva	422	Regla de negocio → "La operación no cumple una regla del inventario"
Error interno	500	GlobalExceptionHandler → "El servidor tuvo un problema"
Respuesta con forma inesperada	200 pero la fila muestra vacíos o la app falla al renderizar	Nadie lo valida en runtime (TS no existe en ejecución): se ve en Response de Network; un render roto lo atrapa el ErrorBoundary

Muéstralo en el código

- `ProductoTable.tsx:88` → `ProductosPage.tsx:110-120` → `ProductoForm.tsx:90-95`, `:97-125` → `productoService.ts:16-19` → `apiClient.ts:52-68` → backend `ProductoController` / `ProductoService` → `ProductosPage.tsx:122-126` → `ProductoTable.tsx:56`.

Si el POST aparece 201 en Network y PostgreSQL tiene el registro, pero la pantalla no lo muestra, ¿dónde termina el problema y cómo lo demostrarías?

El backend y la base cumplieron: el problema está **en el frontend, después de la respuesta**. Lo demuestro por partes: 1) Network → Response trae el objeto con `id` (el contrato está bien); 2) con React DevTools o un `console.log` en `handleSaved` veo si se llamó a `onSaved`; 3) reviso que `setData` reciba un array nuevo; 4) reviso filtros y paginación (el producto puede estar en otra página o fuera del filtro de categoría activo); 5) recargo la página: si aparece, confirma que el dato existe y que falló la sincronización del estado local, no la persistencia.

Los 5 desafíos prácticos

No sustituyen las 50 preguntas: sirven para comprobar que dominas el código y no sólo el discurso.

Desafío 1 — Trazado en vivo

Elegir un POST o PUT real, abrir Page, Form, Hook/Service, apiClient y Network; predecir URL, payload, status y cambio de estado; ejecutar y comprobar.

- Abre en este orden: `ProductosPage.tsx` → `ProductoForm.tsx` → `productoService.ts` → `apiClient.ts` y la pestaña Network.
- **Predicción para "Nuevo producto"**: `POST http://localhost:8080/api/productos`, precedido de un `OPTIONS`; payload `{ categoriaId: 1, codigo: "PRD-OFI-099", nombre: "...", descripcion: null, unidadMedida: "UNIDAD", stockMinimoDefault: 5, precioReferencial: null }` (números, no strings); respuesta **201** con el objeto y su `id`; el estado `productos.data` crece en uno, el modal se cierra y aparece un toast.
- Ejecuta y compara cada punto en Network (Headers, Payload, Response).

Desafío 2 — Romper una pieza

Si…	…entonces (predicción)
Quitas `reloadToken` de las dependencias del Effect de `useAsyncList`	"Reintentar" y los `reload()` tras un 404 dejan de recargar: el estado cambia pero el Effect no se vuelve a ejecutar. (La regla `exhaustive-deps` de los linters de React advierte este tipo de omisión.)
Cambias `rowKey={(p) => p.id}` por el índice	Al borrar o filtrar, React reutiliza filas equivocadas: el menú de acciones abierto o el estado "cargando" (`busyId`) queda en otra fila.
Borras `VITE_API_URL` de `.env.development`	La app no arranca: `config/env.ts` lanza "Falta VITE_API_URL: crea .env.development…" (hay que reiniciar `npm run dev` para que Vite relea el `.env`).
Mueves la llamada HTTP a `ProductoTable`	Funciona, pero la tabla queda acoplada al backend, la Page no se entera del borrado, el manejo del 409 y los toasts se duplican, y las pruebas de la tabla necesitarían simular HTTP.

Desafío 3 — Error intencional

Provocar	Network	Capa que lo detecta	Qué ve el usuario
URL incorrecta dentro de `/api` (en el service, `'/producto'` en lugar de `'/productos'`)	404 en `/api/producto`	Backend (sin mapping) → `ApiError`	Alert "No se pudieron cargar los productos" con el mensaje del backend + Reintentar
Prefijo incorrecto (`VITE_API_URL=http://localhost:8080/apix`)	404, pero marcado como error CORS: fuera de `/api/**` el backend no agrega cabeceras CORS	Navegador → `TypeError`	"No se pudo conectar con el backend…" (por eso conviene mirar Network y no sólo el mensaje)
Backend apagado	Petición fallida (`ERR_CONNECTION_REFUSED`), sin status	Navegador → `TypeError`	"No se pudo conectar con el backend. ¿Está corriendo StockFlowApplication…?"

Payload inválido (código vacío)	Ninguna petición	Validación de UI	"El código es obligatorio" junto al campo, con foco en él
Id inexistente (editar un producto borrado por otro)	`GET /productos/{id}` → 404	Backend	Toast "El registro ya no existe… Se actualizó la lista" y recarga
Conflicto 409 (código `PRD-OFI-001` repetido; borrar `CAT-OFI`)	POST o DELETE → 409	Backend (regla + UNIQUE/FK)	Mensaje del backend; el formulario o la fila se conservan

Desafío 4 — Relación y estado

- Edita un producto y cámbiale la categoría → guarda → la columna Categoría cambia sin recargar, porque se resuelve con `categoriasPorId.get(p.categoriaId)` y no se duplica el nombre.
- Aplica el filtro por la categoría vieja: el producto ya no aparece. Por la nueva: aparece.
- Explica el orden de derivación: `productos.data` → texto + categoría → estado → `paginar` → tabla; y que cambiar un filtro vuelve a la página 1.

Desafío 5 — Producción y pruebas

- Ejecuta `npx vitest run src/features/categorias/pages/CategoriasPage.test.tsx` y explica una prueba de interacción (por ejemplo, la del 409 en DELETE).
- `npm run build` → inspecciona `dist/index.html` y `dist/assets` (un `.js` por página lazy, nombres con hash) y busca la URL embebida de la API.
- Explica que el hosting necesita el fallback a `index.html` para `BrowserRouter` (en StockFlow, `nginx.conf`: recarga `/productos` con F5 sobre nginx) y que, con el proxy `/api/`, frontend y API comparten origen; si la API estuviera en otro dominio, el backend debería permitir el del frontend en `CORS_ALLOWED_ORIGINS`.

Señales de alerta y checklist final

Todavía NO estás preparado si…

- Puedes copiar componentes, pero no sabes explicar el recorrido desde un clic hasta el re-render.
- Confundes la ruta `/productos` de React Router con el endpoint `/api/productos`.
- Dices que TypeScript valida automáticamente el JSON recibido en tiempo de ejecución.
- No sabes por qué un input es controlado ni qué hacen `value`, `onChange` y `useState`.
- No puedes explicar por qué `ProductoFormData` usa string y el payload number.
- La tabla llama directamente al service y no puedes justificar el acoplamiento.
- No distingues error de red, error HTTP, CORS y error de renderizado.
- No sabes por qué hay que revisar `response.ok` con `fetch`.
- No puedes explicar la relación Categoría 1:N Producto: select → `categoriaId` → payload → respuesta → UI.
- Usas `useEffect` como "ejecutar al montar" y no sabes explicar dependencias ni cleanup.
- Desactivarías StrictMode para ocultar la doble ejecución sin entender la causa.
- Guardas listas filtradas o paginadas como estados duplicados.
- No puedes diagnosticar una petición real usando DevTools/Network.
- No distingues pruebas de UI de pruebas de integración con la API.
- No sabes que `dist` es un build estático ni por qué `BrowserRouter` necesita fallback SPA.

Checklist personal

☐ Puedo explicar la estructura completa del frontend y la responsabilidad de cada carpeta.
☐ Puedo recorrer `index.html` → `main.tsx` → `App` → `AppRouter` → `MainLayout` → Page.
☐ Puedo explicar props, estado, render, callbacks y listas con key estable.
☐ Puedo construir y defender `CategoriaForm` y `ProductoForm` como formularios controlados.
☐ Puedo explicar cómo convierto los datos del formulario al payload que espera el backend.
☐ Puedo dibujar GET, POST, PUT y DELETE desde React hasta el backend y de regreso.
☐ Puedo diferenciar CORS, 400, 404, 409, 422, 500 y error de red con ejemplos de StockFlow.
☐ Puedo explicar la relación Categoría 1:N Producto y el cambio de categoría de un producto.
☐ Puedo explicar `useEffect`, dependencias, cleanup, `AbortController` y StrictMode.
☐ Puedo justificar los custom hooks, el estado derivado, la búsqueda, los filtros y la paginación.
☐ Puedo defender `Modal`/`ConfirmDialog`, la accesibilidad y los componentes UI compartidos.
☐ Puedo ejecutar y explicar las pruebas con Vitest y Testing Library.
☐ Puedo generar el build de producción y explicar `dist`, `preview`, `.env` por modo y fallback SPA.
☐ Puedo usar Network y Console para encontrar la capa exacta donde falla un flujo.
☐ Puedo defender una operación fullstack completa sin mirar apuntes.

Pendientes conocidos del frontend

Puntos que el docente podría señalar. Conviene conocerlos y saber explicarlos con honestidad (la guía formal pide documentar la deuda técnica). Los del fallback SPA y la URL de producción (preguntas 47 y 48) ya se corrigieron con `nginx.conf` y `VITE_API_URL=/api`.

#	Pendiente	Pregunta	Cómo explicarlo o corregirlo
1	No hay pruebas automáticas contra el backend real	43, 45	Las pruebas de páginas simulan los services. La integración se verifica con Postman, las pruebas Testcontainers del backend y Network. Mejora posible: MSW o una prueba e2e del flujo principal.
2	El examen menciona `npm run backend:check`, que no existe en StockFlow	23	El equivalente es `healthService.comprobar()` (`GET /api/health`), que usa la pantalla de acceso, o `curl http://localhost:8080/api/health`.
3	`ProductoForm` y `CategoriaForm` llaman al service directamente (la tabla no)	15, 24	Es el patrón de las guías del docente: en G06 `ClienteForm` llama a `clienteService.crear/actualizar` y avisa con `onSaved(cliente, mode)`, y en G07 `VehiculoForm` hace lo mismo. El formulario es dueño de su envío (validar, guardar, mostrar errores por campo) y la Page decide qué hacer con el resultado. La tabla, en cambio, es pura presentación y no importa services.
4	`NotificationsMenu` (en `components/common`) consulta `inventarioService`	01, 08	Es una pieza del marco que funciona como mini-feature (alertas de stock en el Header). Si crece, conviene moverla a `features/inventario/components`.
5	Login, Perfil, Usuarios y Configuración son demostración (no hay autenticación en el backend)	—	Están rotuladas como DEMO en el menú y no hay protección de rutas a propósito, para no aparentar una seguridad que no existe (comentario en `AppRouter.tsx:21-23`).
6	No hay validación del JSON en tiempo de ejecución	04, 50	`apiFetch` confía en el contrato (`as T`). Mejora posible: validar respuestas críticas con un esquema (por ejemplo Zod).
7	Sin control de concurrencia optimista (`@Version`)	26, 27	Si dos usuarios editan el mismo registro, el último PUT pisa al primero (actualización perdida). Pedir `GET /{id}` al abrir el formulario lo reduce; la solución es un `@Version` en el backend que responda 409 si el registro cambió.
8	Ubicaciones sin PUT ni DELETE	26-28	La API sólo lista, busca y crea (RF-07). La pantalla Almacenes lo avisa. Si el docente pide editar o borrar una ubicación, responder con Categoría/Producto y explicarlo como pendiente del backend.
9	El Dashboard calcula sus métricas en el navegador con las colecciones completas	37, 49	Sirve con pocos datos. Con volumen, lo correcto es un endpoint agregado (`GROUP BY` en PostgreSQL); el comentario de `DashboardPage` lo anticipa.

Glosario de términos técnicos

Cada término está explicado de forma simple y relacionado con StockFlow. Entre corchetes, las preguntas donde aparece.

A

AbortController / AbortSignal

API del navegador para **cancelar** operaciones asíncronas como `fetch`. Se crea un `AbortController`, se pasa su `signal` a `fetch` y, al llamar `controller.abort()`, la petición se corta y su Promise se rechaza con un `AbortError`. En StockFlow se usa en el cleanup de `useAsyncList` y `useKardex` para que una pantalla cerrada o una petición vieja no actualicen el estado. [39, 38, 40]

AbortError

El error que produce una petición cancelada. **No es un fallo del sistema** (fue intencional), por eso `isAbortError()` lo reconoce y no se muestra ningún mensaje. [39, 22]

Accesibilidad (a11y)

Que la aplicación pueda usarla cualquier persona: con teclado, con lector de pantalla, con baja visión. Se logra con etiquetas (`label`), roles, atributos `aria-*`, foco bien manejado, botones reales y buen contraste. "a11y" es la abreviatura en inglés (a + 11 letras + y). [20, 42, 44]

Adapter (adaptador)

Pieza que traduce entre dos formatos para que el resto del código no dependa del exterior. En el frontend sería la función que convierte la respuesta del backend al modelo de la UI; si el backend renombra un campo, sólo se toca el adapter. [05]

API / API REST

API (interfaz de programación) es el conjunto de operaciones que un sistema ofrece a otros. Una **API REST** las ofrece por HTTP usando recursos con URL (`/api/productos`) y métodos (GET, POST, PUT, DELETE). El backend de StockFlow es una API REST que el frontend consume. [06, 21]

apiClient / apiFetch

El módulo `src/api/apiClient.ts`: la **única puerta HTTP** del frontend. Agrega la URL base y los headers JSON, llama a `fetch`, revisa `response.ok`, convierte errores en `ApiError` y trata el 204. [21, 22, 24]

ApiError

Clase de error propia de StockFlow (extiende `Error`) que lleva el `status` HTTP y los `fieldErrors` enviados por el backend. Permite que las pantallas distingan un 404 de un 409 sin leer la respuesta. [22, 29]

aria-* (atributos ARIA)

Atributos HTML que describen elementos a los lectores de pantalla: `aria-label` (nombre), `aria-labelledby`/`aria-describedby` (enlazan con un título o una ayuda), `aria-invalid` (campo con error), `aria-current` (página actual), `aria-busy` (cargando), `aria-hidden` (decorativo). [20, 42]

async / await

`async` marca una función que devuelve una Promise. Dentro de ella, `await` espera el resultado de otra Promise **sin bloquear el navegador**: la función queda en pausa y el resto de la página sigue funcionando. [31]

Asíncrono

Una operación que termina "más tarde" (una petición de red, un temporizador). El programa no se queda esperando: sigue con otras cosas y procesa el resultado cuando llega. [31, 38]

B

Backend
La parte del sistema que corre en el servidor: reglas de negocio, acceso a la base de datos y API. En StockFlow es Spring Boot (Java 21) con PostgreSQL. El frontend nunca habla con la base directamente: siempre pasa por el backend. [21, 50]

Batching
React agrupa varias actualizaciones de estado que ocurren juntas (por ejemplo `setData` y `setLoading`) en un solo re-render, para no dibujar la pantalla varias veces. [21]

Bean Validation (@Valid)
Mecanismo del backend Java que valida los datos de entrada con anotaciones (`@NotBlank`, `@Size`…). Si fallan, el backend responde 400 con los errores por campo. [14, 29]

Body (cuerpo)
El contenido de una petición o respuesta HTTP. En StockFlow es JSON. GET y DELETE normalmente no llevan body; POST y PUT sí. Una respuesta 204 no tiene body. [21, 25, 28]

BrowserRouter
Componente de React Router que usa la History API del navegador para manejar URLs "limpias" (`/productos`). Requiere fallback SPA en producción. [06, 48]

Build
El proceso que transforma el código fuente en los archivos finales que se publican. En StockFlow, `npm run build` verifica los tipos con `tsc` y empaqueta con Vite en `dist/`. [02, 47]

Bundle / bundler
Un **bundle** es un archivo JavaScript que junta muchos módulos. El **bundler** (Vite usa Rollup/Rolldown en el build) los une, elimina lo que no se usa y los minifica para que el navegador descargue menos. [02, 47]

C

Caché y hash en nombres
El navegador guarda archivos para no descargarlos de nuevo. Vite agrega un hash al nombre (`apiClient-VTyo5whh.js`) que cambia cuando cambia el contenido; así una versión nueva nunca se confunde con una vieja guardada. [47]

Callback
Una función que se pasa a otro componente o función para que la "llame de vuelta" cuando ocurra algo. En React, los callbacks por props (`onEdit`, `onSaved`, `onConfirm`) permiten que un hijo avise al padre. [10, 15]

Cleanup
La función que devuelve un `useEffect`. React la ejecuta antes de repetir el Effect y al desmontar el componente, para deshacer lo que hizo el setup (cancelar peticiones, temporizadores o suscripciones). [38, 39, 40]

Code splitting / lazy loading
Dividir el JavaScript en trozos y descargar cada uno sólo cuando hace falta. StockFlow usa `lazy(() => import(...))` en las páginas: el código de Productos se descarga la primera vez que se visita. Mientras llega, `Suspense` muestra un esqueleto. [02, 06]

Código de estado HTTP (status)

Número que indica el resultado de una petición. 2xx = éxito (200 OK, 201 Created, 204 No Content); 4xx = error del cliente (400 datos inválidos, 404 no existe, 409 conflicto, 422 regla de negocio); 5xx = error del servidor (500). [22, 29]

Componente

Función de React que recibe props y devuelve la interfaz (JSX). Es la pieza básica de la UI: `Button`, `ProductoTable`, `ProductosPage`. [08, 10]

Componente (input) controlado

Un control de formulario cuyo valor sale del estado de React (`value`) y se actualiza con `onChange`. El estado es la fuente de verdad, no el DOM. El opuesto es el no controlado, que guarda su propio valor (`defaultValue`). [11, 34]

Constraint (restricción)

Regla que la base de datos hace cumplir siempre: `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `NOT NULL`. Es la última línea de defensa de la integridad. [14, 33]

Contrato (HTTP)

El acuerdo entre frontend y backend: qué URL, método, campos, tipos, status y formato de error tiene cada operación. Los modelos TypeScript del frontend reflejan ese contrato (los DTO), no la base de datos. [05, 43, 45]

Context (React)

Mecanismo para compartir un valor con muchos componentes sin pasarlo por props en cada nivel. StockFlow lo usa para el tema, la sesión demo y los toasts (los providers de `App.tsx`). [03, 41]

CORS (Cross-Origin Resource Sharing)

Regla de seguridad **del navegador**: una página sólo puede leer respuestas de otro origen si ese servidor lo autoriza con la cabecera `Access-Control-Allow-Origin`. Postman no la aplica porque no es un navegador. [23, 32, 48]

CRUD

Las cuatro operaciones básicas sobre datos: **C**reate (POST), **R**ead (GET), **U**pdate (PUT) y **D**elete (DELETE). [25-28]

D

Design tokens

Variables CSS con nombres de significado (`--surface`, `--text-muted`, `--brand`) en lugar de colores o medidas sueltas. Todos los componentes las usan, así el diseño es coherente y el tema oscuro se logra cambiando sólo los valores. En StockFlow: `styles/tokens.css`. [42]

DevTools (Network, Console)

Herramientas del navegador (F12). **Network** muestra cada petición real (URL, método, status, headers, payload, respuesta, tiempo); **Console** muestra errores y avisos. Son la evidencia para diagnosticar. [21, 32, 50]

dist/

Carpeta que genera `npm run build`: `index.html` y `assets/` con el JavaScript y CSS finales. Es lo que se sube al hosting. No se versiona en Git. [47]

DOM

(Document Object Model) La representación en memoria de la página que el navegador muestra. React calcula qué debe cambiar y actualiza el DOM por nosotros. [03, 11]

DTO (Data Transfer Object)

Objeto pensado sólo para transportar datos entre sistemas (por ejemplo `ProductoResponse` o `CrearProductoRequest` del backend). Define la forma del JSON, separada de cómo se guarda en la base. [05, 35]

E

Effect / useEffect

Hook para **sincronizar** un componente con algo externo a React (API, temporizador, foco, `<dialog>`). Tiene setup, dependencias y cleanup. Se ejecuta después del render cuando cambian sus dependencias. [38, 39, 40]

Endpoint

Una dirección concreta de la API con un método: `GET /api/productos`, `PUT /api/productos/{id}`. No confundir con una ruta del navegador. [06, 32]

Entidad JPA (@Entity)

Clase del backend que representa una tabla de la base de datos (`ProductoJpaEntity`). Es un detalle interno del backend; el frontend no debe copiarla. [05]

Error Boundary

Componente que atrapa errores ocurridos **durante el render** de sus hijos y muestra una pantalla alternativa. No atrapa errores de `fetch` ni de handlers de eventos. En StockFlow: `AppErrorBoundary`. [46]

Error de red

Cuando la petición no obtiene ninguna respuesta (servidor apagado, sin conexión, CORS bloqueado). `fetch` lanza `TypeError`. Es distinto de un error HTTP, que sí trae status. [22, 23]

Estado (state)

Datos que pertenecen a un componente y que, al cambiar, hacen que React vuelva a dibujarlo. Se crea con `useState` y sólo se cambia con su setter. [10, 11, 17]

Estado fuente / estado derivado

Fuente: lo mínimo que hay que guardar (datos del backend, lo que eligió el usuario). **Derivado**: lo que se calcula a partir de la fuente (lista filtrada, página, totales). Lo derivado no se guarda en `useState` para que nunca quede desincronizado. [17, 19, 49]

Event loop

El mecanismo del navegador que procesa tareas una por una (eventos, respuestas de red, renders). Gracias a él un `await` no congela la página: mientras se espera, se atienden otras tareas. [31]

Evento / evento tipado

Aviso que genera el navegador cuando el usuario interactúa (clic, tecla, envío). En TypeScript se tipa: `ChangeEvent<HTMLInputElement>`, `FormEvent<HTMLFormElement>`. [13]

F

Fallback SPA

Regla del servidor web que devuelve `index.html` para cualquier ruta que no sea un archivo real, para que React Router pueda resolver URLs como `/productos` al entrar directo o al recargar. [48]

Feature (carpeta)

Forma de organizar el código por capacidad de negocio: todo lo de productos (modelos, service, hooks, componentes, páginas) vive en `features/productos/`. [01, 08]

fetch
Función del navegador para hacer peticiones HTTP. Devuelve una Promise con un `Response`. Sólo rechaza ante errores de red o aborto; un 404 o un 500 "resuelven" igual. [21, 22]

fieldErrors
Objeto que envía el backend en un 400 con el error de cada campo (`{ codigo: "no debe estar vacío" }`). El formulario lo usa para mostrar cada mensaje junto a su input. [14, 22, 25]

FK (clave foránea)
Columna que apunta a la clave primaria de otra tabla y que la base obliga a ser válida. `producto.categoria_id` es la FK hacia `categoria`: materializa la relación 1:N. [33, 35]

FormData (en StockFlow)
Tipo del estado de un formulario (`ProductoFormData`): representa los datos **mientras se editan**, con números como texto. No confundir con la clase `FormData` del navegador, que StockFlow no usa. [05, 12]

Frontend
La parte del sistema que ve y usa la persona: corre en el navegador. En StockFlow, React + TypeScript + Vite. [01]

G - H

Genérico (<T>)
Un "tipo comodín" que se completa al usarlo. `useAsyncList<T>` funciona para cualquier entidad: con `T = Producto` devuelve `Producto[]`. Evita escribir lo mismo para cada tipo. [41]

Header (cabecera HTTP)
Información extra de una petición o respuesta: `Content-Type` (formato del body), `Accept` (formato que acepto), `Origin` (desde dónde se pide), `Location` (URL del recurso creado). [21, 23, 25]

HMR (Hot Module Replacement)
Función del servidor de desarrollo de Vite: al guardar un archivo, reemplaza sólo ese módulo en el navegador sin recargar la página, conservando casi siempre el estado. [02]

Hook
Función de React que empieza con `use` y da capacidades a un componente: `useState` (estado), `useEffect` (sincronización), `useMemo` (memorizar cálculos), `useId` (ids únicos), `useCallback` (funciones estables). Sólo se llaman en el nivel superior de un componente o de otro hook. [11, 38, 41]

Hook personalizado (custom hook)
Función propia que agrupa hooks para reutilizar lógica: `useAsyncList`, `useProductos`, `useKardex`. Comparte la lógica, no el estado: cada llamada tiene el suyo. [41]

Hosting
El servidor o servicio donde se publica la aplicación (nginx, Vercel, Netlify…). Para un frontend Vite basta con servir los archivos de `dist/` con fallback SPA. [47, 48]

HTTP y sus métodos
Protocolo de petición/respuesta de la web. Métodos: **GET** (leer), **POST** (crear), **PUT** (reemplazar), **DELETE** (borrar), **OPTIONS** (preguntar qué se permite; lo usa el preflight de CORS). [21-28]

I - J - K

Idempotencia
Una operación es idempotente si repetirla deja el mismo resultado que hacerla una vez. PUT y DELETE lo son; POST no (dos POST crean dos registros o el segundo da 409). [27]

import.meta.env
Objeto donde Vite expone las variables de entorno al código. Sólo incluye las que empiezan con `VITE_` (más algunas propias como `MODE`). Su valor se incrusta al compilar. [09, 47]

Inmutabilidad
No modificar un objeto o array existente, sino crear uno nuevo con el cambio (`{ ...prev, campo }`, `[...prev, nuevo]`, `map`, `filter`). React detecta cambios comparando referencias; si mutas, puede no re-renderizar. [11, 16, 25]

Integración (prueba de)
Prueba que verifica que varias piezas reales funcionan juntas, por ejemplo el frontend contra el backend real. Detecta errores de contrato que los mocks no ven. [43, 45]

interface / type
Formas de declarar tipos en TypeScript. `interface` describe la forma de un objeto y se puede extender (`extends`); `type` además permite uniones, tipos derivados y combinaciones. Ambos desaparecen al compilar. [04]

Inversión de control
En vez de que un componente decida todo, recibe desde afuera qué hacer (callbacks). La tabla emite "quieren editar"; la página decide el efecto. [15]

jsdom
Implementación de un navegador en JavaScript, sin pantalla, que usa Vitest para que existan `document` y `window` en las pruebas. No implementa todo (por ejemplo `<dialog>.showModal()`), por eso `test/setup.ts` lo simula. [44]

JSON
Formato de texto para datos (`{"id": 1, "nombre": "Resma"}`). Viaja en el body de las peticiones; `JSON.stringify` convierte objetos a texto y `response.json()` hace lo contrario. [21, 25]

JSX / TSX
Sintaxis que permite escribir algo parecido a HTML dentro de JavaScript (`<Button>Guardar</Button>`). TSX es lo mismo en archivos TypeScript. Vite la transforma en llamadas de JavaScript. [02, 03]

Kardex
Registro histórico de todos los movimientos de inventario de un producto (entradas, salidas, transferencias) con el saldo resultante. En StockFlow se consulta paginado en el servidor desde la pantalla Movimientos. [37, 39]

key
Identificador único que React usa para reconocer cada elemento de una lista entre renders. Debe ser estable (el `id`), no el índice. [16]

L - M

Layout
Estructura fija que envuelve varias páginas (menú, encabezado). En StockFlow, `MainLayout`, con un `Outlet` donde se pinta la página actual. [06, 08]

Link / NavLink
Componentes de React Router para navegar sin recargar. `NavLink` además sabe si su ruta está activa (`isActive`) para resaltarla en el menú. [07]

Linter
Herramienta que revisa el código buscando errores y malas prácticas sin ejecutarlo. StockFlow usa `oxlint` (`npm run lint`), con reglas de hooks de React. [38]

Map
Estructura de JavaScript que guarda pares clave → valor y busca por clave al instante. `categoriasPorId` es un `Map<number, Categoria>`. [36]

map / filter / find
Métodos de arrays. `map` transforma cada elemento; `filter` se queda con los que cumplen una condición; `find` devuelve el primero que la cumple. Ninguno modifica el array original. [16]

Minificar
Reducir el tamaño del código quitando espacios, comentarios y acortando nombres, sin cambiar lo que hace. Lo hace el build. [02, 47]

Mock
Reemplazo falso de una dependencia en una prueba, que responde lo que el test decide. StockFlow simula los services con `vi.mock` y `fetch` con `vi.stubGlobal`. [43, 45]

Modal / <dialog>
Ventana que aparece encima de la página y bloquea el fondo hasta cerrarse. StockFlow usa el elemento nativo `<dialog>` con `showModal()`, que ya maneja foco, Escape y fondo inerte. [20, 42]

Modo (development / production / test)
Vite carga un archivo `.env` distinto según el modo: `npm run dev` → development, `npm run build` → production, Vitest → test. [09, 47]

MSW (Mock Service Worker)
Librería que intercepta las peticiones a nivel de red durante las pruebas y responde datos simulados. Permite probar también el cliente HTTP y las URLs. StockFlow no la usa. [45]

N - O

node_modules / package.json / package-lock.json
`package.json` lista las dependencias y los scripts; `package-lock.json` fija las versiones exactas instaladas; `node_modules/` contiene las librerías descargadas (se regenera con `npm install` y no se sube a Git). [02]

npm
El gestor de paquetes de Node.js: instala librerías (`npm install`) y ejecuta los scripts de `package.json` (`npm run dev`). [02]

Origen (origin)
La combinación de **esquema + host + puerto** de una URL. `http://localhost:5173` y `http://localhost:8080` son orígenes distintos porque cambia el puerto. [23]

Outlet
Componente de React Router que marca el lugar dentro de un layout donde se pinta la ruta hija que coincide con la URL. [06]

P

Page (página)
Componente que representa una pantalla completa y coordina su lógica: carga datos, maneja estado y decide qué hacer en cada acción (`ProductosPage`). [08, 10]

Paginación (cliente / servidor)
Dividir una lista en páginas. **En el cliente**: se descargan todos los datos y se cortan con `slice` (Productos). **En el servidor**: se pide sólo una página y el backend informa el total (Kardex). [19, 37]

Payload
Los datos que se envían en el body de una petición (el JSON de un POST o PUT). En StockFlow se arma con `aPayload` después de validar. [12, 25, 32]

Preflight
Petición `OPTIONS` automática que el navegador envía antes de ciertas peticiones entre orígenes (por ejemplo, POST/PUT con JSON) para preguntar si están permitidas. Si falla, la petición real no se envía. [23, 25]

preventDefault()
Método de un evento que cancela el comportamiento por defecto del navegador; en un formulario, evita la recarga de la página al enviarlo. [13]

Preview
`npm run preview`: sirve localmente la carpeta `dist/` para revisar el build antes de publicarlo. No es un servidor de producción. En StockFlow reenvía `/api` al backend local. [47]

Promise
Objeto que representa un resultado futuro: queda **pendiente**, luego se **cumple** con un valor o se **rechaza** con un error. `fetch` y las funciones `async` devuelven Promises. [22, 31]

Props
Los parámetros que un componente padre le pasa a un hijo. Son de solo lectura para el hijo. Pueden ser datos o funciones (callbacks). [10, 15]

Provider
Componente que pone un valor de Context a disposición de todos sus descendientes (`ThemeProvider`, `ToastProvider`). [03]

Proxy inverso
Servidor que recibe las peticiones del navegador y las reenvía a otro servidor. En StockFlow, `nginx.conf` sirve `dist/` y reenvía `/api/` al backend: el navegador ve un solo origen, por eso `.env.production` usa `/api` y no hay CORS. [47, 48]

Q - R

Query params / search params
La parte de la URL después de `?` (`/productos?categoria=2`). Sirven para filtros. React Router los lee y escribe con `useSearchParams`. [37]

Race condition (condición de carrera)
Error que depende del orden en que terminan operaciones simultáneas: por ejemplo, una respuesta vieja que llega tarde y pisa a una más nueva. Se evita cancelando (AbortController) y descartando la respuesta si su petición ya fue abortada; en StockFlow lo prueban `useAsyncList.test.ts` y `useKardex.test.ts`. [39, 43]

React
Librería para construir interfaces con componentes. Tú describes cómo debe verse la pantalla según el estado y React se encarga de actualizar el DOM cuando el estado cambia. [todas]

React DevTools
Extensión del navegador para inspeccionar el árbol de componentes, sus props, estado y hooks en vivo. [11, 50]

React Router
Librería que asocia URLs con componentes dentro de una SPA (`BrowserRouter`, `Routes`, `Route`, `Link`, `Outlet`, `useNavigate`). [06, 07]

React Testing Library (RTL)
Librería para probar componentes como los usaría una persona: los busca por rol, etiqueta o texto, y comprueba lo que se ve. [43, 44]

Refetch
Volver a pedir los datos al servidor para que la pantalla coincida con él. En StockFlow, `reload()`. [30]

Regresión
Un error nuevo en algo que antes funcionaba, causado por un cambio. Las pruebas automáticas sirven para detectarlas. [43]

Render / re-render
Cuando React ejecuta un componente para calcular cómo debe verse. Hay un nuevo render cada vez que cambian su estado o sus props. [11, 21]

renderHook
Función de Testing Library que monta un custom hook dentro de un componente de prueba para comprobar su estado (`result.current`) sin una pantalla real. La usan `useAsyncList.test.ts` y `useKardex.test.ts`. [43, 44]

response.ok
Propiedad del `Response` de `fetch`: es `true` si el status está entre 200 y 299. Hay que revisarla porque `fetch` no rechaza ante un 404 o un 500. [22]

Route / Routes
`<Route path="productos" element={...}>` asocia una ruta con un componente; `<Routes>` elige la que coincide con la URL actual. [06]

Ruta del navegador vs endpoint
La ruta del navegador (`/productos`) decide qué **pantalla** se ve y la resuelve React Router. El endpoint (`/api/productos`) devuelve **datos** y lo resuelve el backend. [06, 32]

Runtime vs compilación
Compilación: cuando se revisa y transforma el código (ahí actúa TypeScript). **Runtime** (tiempo de ejecución): cuando el programa corre en el navegador. Los tipos de TS sólo existen en compilación. [04]

S

Secreto
Dato que no debe ser público: contraseñas, tokens, claves privadas. Nunca va en el frontend ni en variables `VITE_`, porque todo lo del frontend es visible para el usuario. [09]

Service (servicio)
Módulo que agrupa las operaciones HTTP de un recurso con nombres de negocio (`productoService.listar`, `crear`…). Es el único que conoce las rutas de ese recurso. [24]

Setter (setState)
La función que devuelve `useState` para cambiar el estado (`setFormData`, `setPage`). Llamarla hace que React agende un nuevo render. Acepta un valor o una función `prev => nuevo`. [11, 21]

SPA (Single Page Application)
Aplicación de una sola página HTML: la navegación cambia la pantalla con JavaScript sin recargar. StockFlow es una SPA. [01, 07, 48]

Stale closure / datos obsoletos
Cuando una función o Effect sigue usando un valor viejo porque lo "capturó" en un render anterior y no se volvió a ejecutar. Suele deberse a dependencias faltantes. [38]

StrictMode
Modo de React sólo para desarrollo que ejecuta dos veces renders y Effects (setup → cleanup → setup) para revelar errores. No afecta producción. [40]

Suspense
Componente de React que muestra un contenido de espera (fallback) mientras algo se carga, por ejemplo el código de una página lazy. [06]

Swagger / OpenAPI
OpenAPI es la descripción estándar de una API; Swagger UI es la página que la muestra y permite probar endpoints. En StockFlow: `http://localhost:8080/swagger-ui.html`. Se sirve desde el mismo backend, por eso no tiene problemas de CORS. [23, 32]

T

Toast
Notificación breve que aparece en una esquina y desaparece sola ("Producto creado"). StockFlow la usa para confirmar operaciones y mostrar errores que no pertenecen a un formulario. [18, 25, 28]

TypeError (en fetch)
Error que lanza `fetch` cuando no hubo respuesta (red, CORS, servidor apagado). StockFlow lo traduce a "No se pudo conectar con el backend". [22, 23]

TypeScript
JavaScript con tipos. Revisa en el editor y al compilar que los datos tengan la forma esperada. En la versión 6 (la de StockFlow) el modo estricto viene activado por defecto. Los tipos se borran al ejecutar. [04, 05]

U - V

UNIQUE
Constraint de la base que impide valores repetidos en una columna (`uq_producto_codigo`). Resuelve la carrera cuando dos usuarios guardan el mismo código a la vez. [14, 29]

useMemo
Hook que memoriza el resultado de un cálculo y sólo lo repite si cambian sus dependencias. Es una optimización: el código debe ser correcto sin él. [36, 42]

useNavigate / useSearchParams / useLocation
Hooks de React Router: navegar desde código, leer/escribir los parámetros `?` de la URL y leer la ubicación actual (incluido el `state` de navegación). [07, 37]

userEvent
Librería de pruebas que simula interacciones reales de un usuario (clic, escribir, tabular) disparando todos los eventos intermedios. [44]

useState
Hook que crea una variable de estado y su setter: `const [page, setPage] = useState(1)`. [11, 17]

Validación (UI / backend / base)
Comprobar que los datos son correctos. En la UI ayuda al usuario; en el backend es la autoridad; en la base (constraints) es la garantía final. [12, 14]

Variables de entorno
Valores de configuración que se definen fuera del código (archivos `.env` o el sistema) para cambiar el comportamiento sin editar el programa. [09, 47]

Vite
Herramienta de desarrollo y build para frontend: servidor con HMR en desarrollo y empaquetador optimizado para producción. [02, 47]

VITE_ (prefijo)
Sólo las variables cuyo nombre empieza con `VITE_` llegan al código del navegador. Es un filtro de seguridad, pero lo que pasa ese filtro es público. [09]

Vitest
Framework de pruebas que usa la configuración de Vite. Ejecuta los archivos `*.test.ts(x)` y ofrece `describe`, `it`, `expect` y `vi` (mocks). [43, 44]

Números de status más usados

200 · 201 · 204
Éxito. 200: OK con datos. 201: recurso creado (trae el objeto y el header `Location`). 204: hecho, sin contenido (DELETE). [21, 25, 28]

400 · 404 · 409 · 422
Errores del cliente. 400: datos mal formados o inválidos. 404: no existe. 409: choca con datos existentes (duplicado, dependencias). 422: válido en forma pero viola una regla de negocio. [29]

500
Error interno del servidor, no previsto. El usuario no puede corregirlo; la app debe avisar sin fingir éxito. [29, 46]